



IIB20303 ASSESSMENT COVERSHEET

Attach this coversheet as the cover of your submission. All sections must be completed.

Section A: Submission Details

- Programme** : Bachelor in Information Technology (Honours) (Internet of Things)
- Course Code & Name** : IIB43203 CLOUD COMPUTING
- Assessment** : Group Project Report
- Dateline** : 7 June 2026
- Penalties** :
- **10%** will be deducted for late submission.
 - Plagiarised work is an Academic Offence in University Rules & Regulations and will be penalised accordingly.
 - **AI content** will affect your mark

Section B: Academic Integrity

Tick (✓) each box below if you agree:

- | | |
|-------------------------------------|--|
| ☒ | I have read and understood the UniKL's policy on Plagiarism in University Rules & Regulations. |
| ☒ | This submission is my own, unless indicated with proper referencing. |
| ☒ | This submission has not been previously submitted or published. |
| ☒ | This submission follows the requirements stated in the course. |

Section C: Submission Detail

Office Receipt of Submission

No	Student Name(s)	Student ID(s)
1.	FARRAH SYAHIRAH SYAHMI BINTI BAHARUDIN	52227124886
2.	NUR HUSNA BINTI MOHAMED DIN	52227124552

3.	NUR NABILAH BINTI MOHD RUZI	52227124109
4.	SITI NURHIDAYAH BINTI ABDULLAH	52227124905

Student Receipt of Submission

The submission receipt of the student task will be based on the **VLE submission date and time.**

Table of Contents

No.	Title	Page
1.	Introduction 1.1 Project Overview 1.2 Project Objectives 1.3 Team Responsibilities 1.4 System Architecture 1.5 System Workflow	5-8
2.	AWS Networking Implementation using Amazon VPC 2.1 Creating Amazon VPC 2.2 Creating Public Subnet 2.3 Enable Auto-Assign Public IPv4 2.4 Creating Internet Gateway 2.5 Create Route Table 2.6 Security Group Verification 2.7 Create Security Group 2.8 Verification	9-25
3.	Web Hosting Implementation using Amazon EC2 3.1 Creating an EC2 Instance 3.2 Configuring Security Group Rules 3.3 Connecting to the EC2 Instance 3.4 Installing Apache Web Server 3.5 Deploying the Website 3.6 Testing Website Accessibility	26-37
4.	Storage Services Implementation using Amazon S3 4.1 Accessing Amazon S3 4.2 Creating the S3 Bucket 4.3 Configuring Bucket Settings 4.4 Enabling Bucket Versioning	38-46

	4.5 Creating Folder Structure 4.6 Uploading Student Files 4.8 Verifying Uploaded Objects 4.9 Role of Amazon S3 in the Student Registration System	
5.	Database Services Implementation using Amazon RDS 5.1 Accessing Amazon RDS 5.2 Creating a MySQL Database Instance 5.3 Configuring Database Coonectivity 5.4 Database Deployment 5.5 Viewing Database Connectivity Details 5.6 Connecting to the Database 5.7 Creating the Student Database and Table 5.8 Inserting and Managing Student Record	47-55
6.	Testing and Validation	56
7.	Project Outcome	57
8.	References	57
9.	Appendix	58

1. Introduction

Cloud computing has transformed the way organizations deploy and manage information systems by providing scalable, reliable, and cost-effective computing resources through the internet. Instead of maintaining physical servers and infrastructure, cloud computing allows resources to be provisioned on demand, reducing operational costs and improving flexibility.

This project focuses on developing an AWS Cloud-Based Student Registration System using Amazon Web Services (AWS). The system demonstrates the integration of cloud services to host a web application, manage databases, provide storage services, and establish secure networking infrastructure.

The project utilizes Amazon EC2, Amazon S3, Amazon RDS, and Amazon VPC to create a complete cloud-based environment. Through this implementation, the project demonstrates how different AWS services can work together to build a secure, scalable, and reliable web application.

1.1. Project Overview

The Student Registration System is a cloud-based web application designed to manage student information efficiently through a web interface.

The system enables users to register student information, store records in a database, and access the information through a hosted website. The project demonstrates the practical application of cloud computing concepts by integrating multiple AWS services into a single solution.

The system architecture consists of:

- Amazon EC2 for web hosting
- Amazon S3 for storage services
- Amazon RDS for database management
- Amazon VPC for networking and security

1.2. Project Objectives

The objectives of this project are:

1. To develop a cloud-based Student Registration System using AWS services.
2. To host the web application using Amazon EC2.
3. To store project files and assets using Amazon S3.
4. To manage student records using Amazon RDS.
5. To implement secure networking infrastructure using Amazon VPC.
6. To demonstrate integration between Compute, Storage, Networking, and Database services.

1.3. Team Responsibilities

The AWS Cloud-Based Student Registration System was developed as a collaborative group project. Each member was assigned a specific AWS service to ensure successful integration of the complete cloud-based application.

Team Member	Responsibility	AWS Service
Farrah Syahirah Syahmi	Networking Infrastructure	Amazon VPC
Siti Nurhidayah	Compute Service	Amazon EC2
Nur Husna	Storage Service	Amazon S3
Nur Nabilah	Database Management	Amazon RDS

The collaboration between all team members ensured that the compute, storage, networking, and database components were successfully integrated into a single cloud-based Student Registration System.

1.4. System Architecture

The overall architecture of the Student Registration System consists of several AWS services working together.

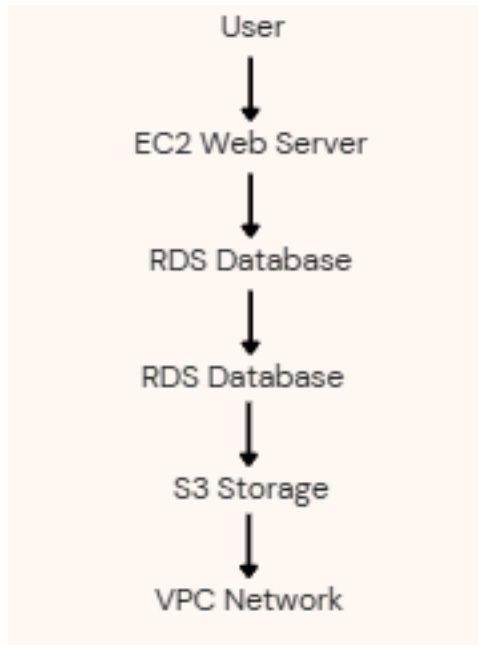


Figure 1: overall architecture of the AWS Cloud-Based Student Registration System

The user accesses the Student Registration System through a web browser. Requests are routed through the Internet Gateway into the Amazon VPC environment. The EC2 instance hosts the web application, Amazon RDS stores student information, and Amazon S3 stores project files and assets.

1.5. System Workflow

The AWS Cloud-Based Student Registration System operates through the integration of several AWS services.

1. A user accesses the Student Registration System through a web browser.
2. The request is sent to the web application hosted on the Amazon EC2 instance.
3. Student information entered through the registration form is processed by the web application.
4. The application stores and retrieves student records from the Amazon RDS database.
5. Files and supporting project assets are stored in Amazon S3.
6. Communication between AWS resources is controlled and secured using Amazon VPC, Route Tables, Internet Gateway, and Security Groups.
7. The user receives the requested information through the website interface.

This workflow demonstrates the integration of Compute, Storage, Networking, and Database services within the AWS cloud environment.

2. AWS Networking Implementation using Amazon VPC

As the team member responsible for networking services, Amazon Virtual Private Cloud (Amazon VPC) was implemented to provide a secure and reliable network infrastructure for the Student Registration System. Amazon VPC was selected because it allows AWS resources to operate within an isolated virtual network while providing controlled communication with the internet and other AWS services.

In this project, Amazon VPC was used to establish the networking environment for the Student Registration System. The network architecture consists of a Virtual Private Cloud, Public Subnet, Internet Gateway, Route Table, and Security Group. These components work together to enable communication between users, the Amazon EC2 web server, and the Amazon RDS database while maintaining network security.

The implementation of Amazon VPC improves the overall system by providing logical network separation, secure access control, and efficient traffic routing. Through this configuration, the Student Registration System can be accessed through the internet while ensuring that communication between AWS resources remains secure and properly managed.

2.1. Step 1: Creating Amazon VPC

- Open AWS Console
- In the search bar, type VPC
- Click VPC
- On the left menu, click Your VPCs

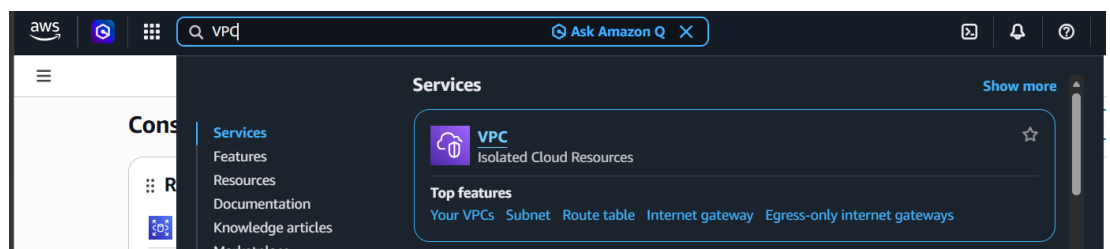


Figure 2: AWS Console search function used to locate and access the Amazon VPC service.

- Click Create VPC

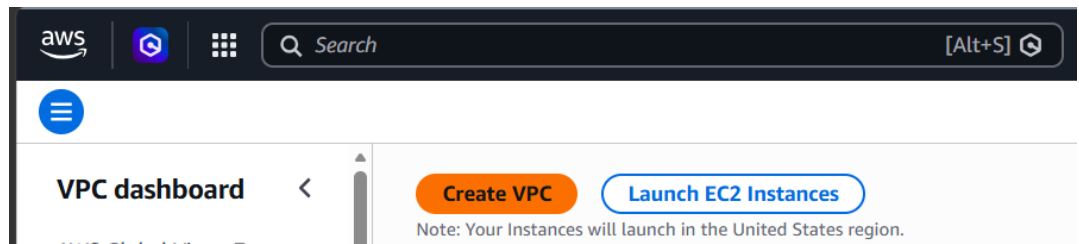


Figure 3: VPC dashboard where a new Virtual Private Cloud can be created.

- Choose VPC only
- Enter:

Parameter	Value
Name tag	Student-vpc
IPv4 CIDR block	10.0.0.0/16
Tenancy	Default

Table 1: Configuration parameters used when creating the Student VPC.

- Click Create VPC

Create VPC [Info](#)

A VPC is an isolated portion of the AWS Cloud populated by AWS objects, such as Amazon EC2 instances.

VPC settings

Resources to create [Info](#)
Create only the VPC resource or the VPC and other networking resources.

☒ VPC only ☐ VPC and more

Name tag - optional
Creates a tag with a key of 'Name' and a value that you specify.

student-vpc

IPv4 CIDR block [Info](#)
☒ IPv4 CIDR manual input
☐ IPAM-allocated IPv4 CIDR block

IPv4 CIDR
10.0.0.0/16
CIDR block size must be between /16 and /28.

IPv6 CIDR block [Info](#)
☒ No IPv6 CIDR block
☐ IPAM-allocated IPv6 CIDR block
☐ Amazon-provided IPv6 CIDR block
☐ IPv6 CIDR owned by me

VPC encryption control (\$) [Info](#)
Monitor mode provides visibility into encryption status without blocking traffic. Enforce mode prevents unencrypted traffic. [Additional charges apply](#)

☒ None ☐ Monitor mode ☐ Enforce mode

Tags
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key **Value - optional**

Q Name X Q student-vpc X [Remove tag](#)

[Add tag](#)
You can add 49 more tags

[Cancel](#) [Preview code](#) [Create VPC](#)

Figure 4: VPC creation settings configured before deployment.

- Outcome

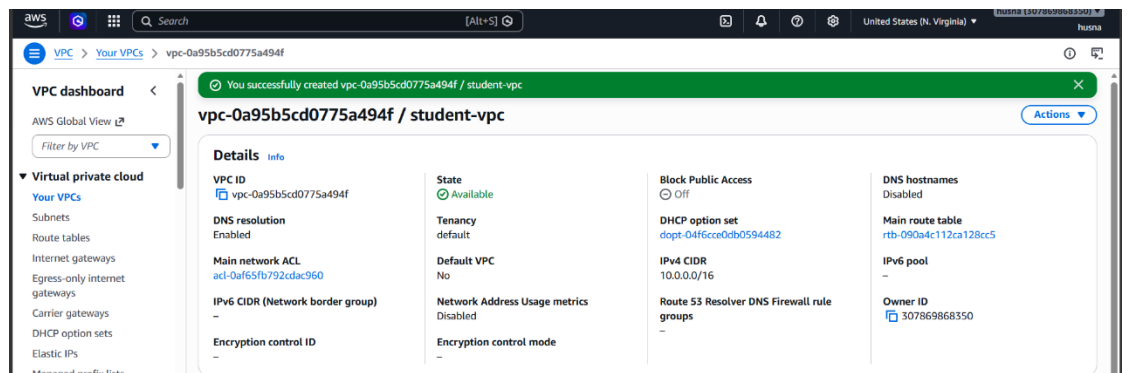


Figure 5: Successfully created Student VPC and its details.

2.2. Creating Public Subnet

- In the VPC dashboard, click Subnets
- Click Create subnet

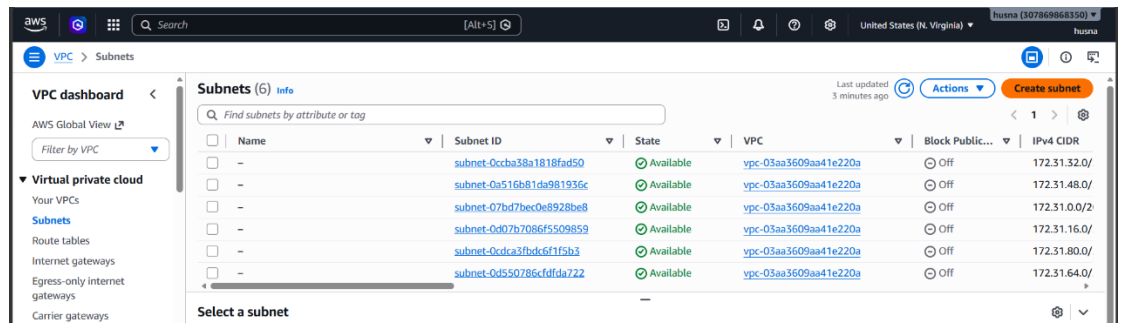


Figure 6: Subnet dashboard used to create a new subnet.

- Under VPC ID, select student-vpc

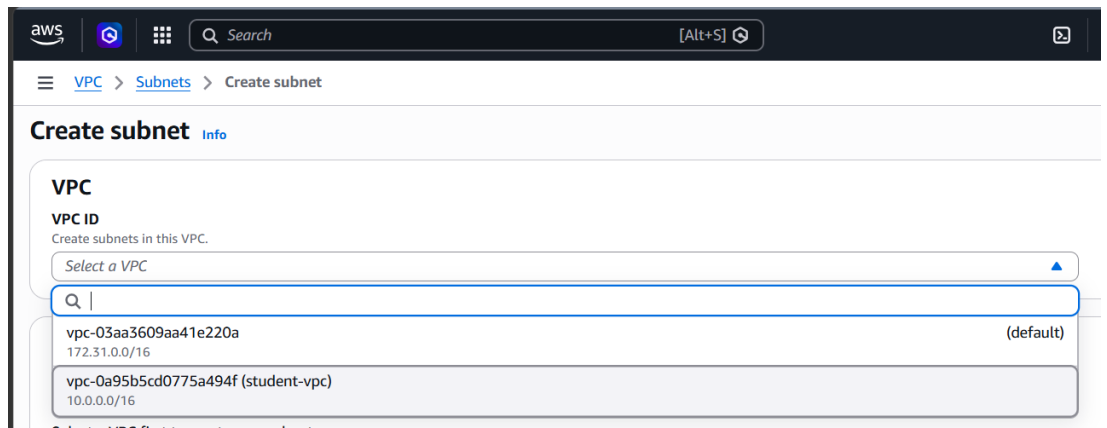


Figure 7: Selection of the Student VPC during subnet creation.

- Under subnet settings, enter:

Parameter	Value
Subnet name	Student-public-subnet
Availability Zone	Use any available zone
Ipv4 subnet CIDR block	10.0.1.0/24

Table 2: The subnet configuration settings used in the project.

- Click Create subnet

Subnet 1 of 1

Subnet name
Create a tag with a key of 'Name' and a value that you specify.
student-public-subnet
The name can be up to 256 characters long.

Availability Zone [Info](#)
Choose the zone in which your subnet will reside, or let Amazon choose one for you.
United States (N. Virginia) / use1-az1 (us-east-1a)

IPv4 VPC CIDR block [Info](#)
Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.
10.0.0.0/16

IPv4 subnet CIDR block
10.0.0.0/24 256 IPs

Tags - optional

Key	Value - optional	
Name	student-public-subnet	Remove

[Add new tag](#)
You can add 49 more tags.

[Remove](#)

[Add new subnet](#)

[Cancel](#) [Create subnet](#)

Figure 8: The subnet configuration page before creation.

- Outcome:

Subnets (1) [Info](#)

[Find subnets by attribute or tag](#)

[Subnet ID: subnet-04fc3e45507f5ba9a](#) [Clear filters](#)

☐	Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR
☐	student-public-subnet	subnet-04fc3e45507f5ba9a	Available	vpc-0a95b5cd0775a494f stud...	Off	10.0.0.0/24

Figure 9: The successfully created public subnet.

2.3. Enable Auto-Assign Public IPv4

- Go to Subnets
- Select student-public-subnet
- Click Actions
- Click Edit subnet settings

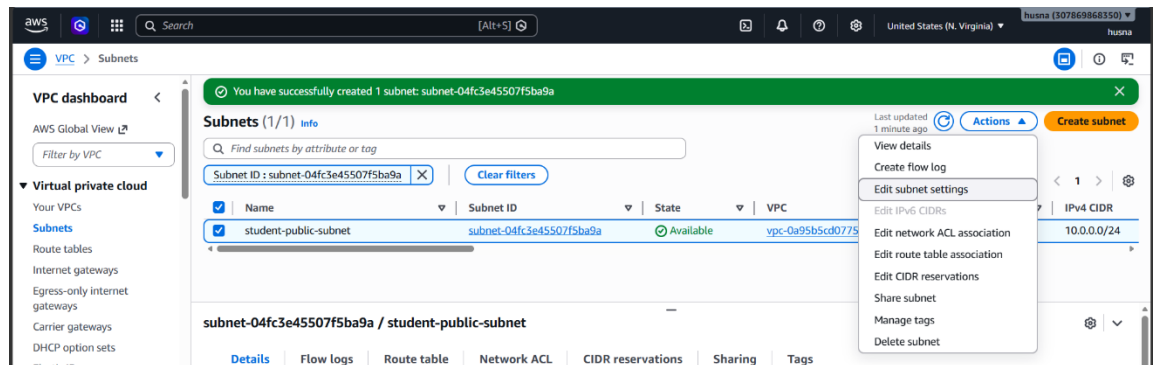


Figure 10: The subnet settings menu used to enable automatic public IP assignment.

- Tick Enable auto-assign public IPv4 address
- Click Save

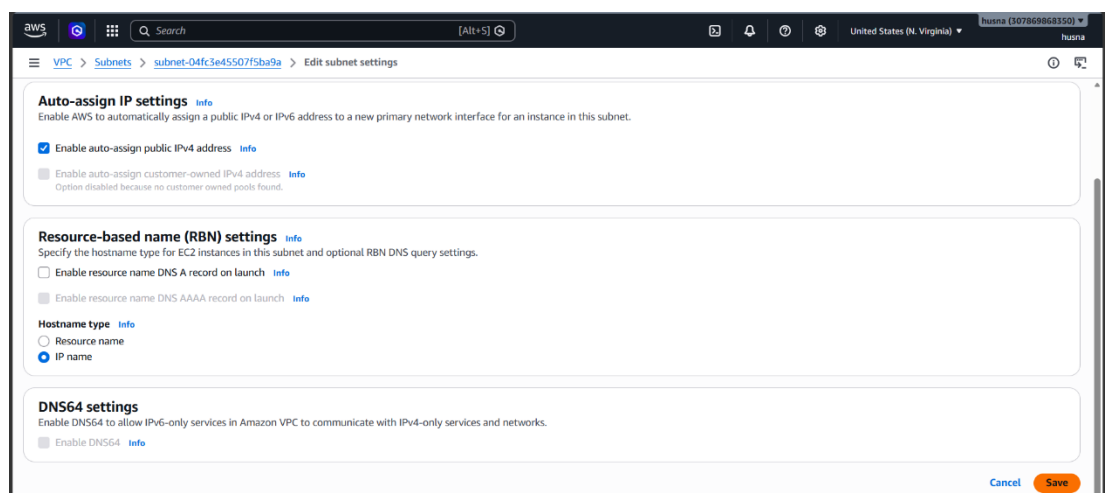


Figure 11: The configuration page for enabling Auto-Assign Public IPv4.

2.4. Creating Internet Gateway

- In the left menu, click Internet gateways
- Click Create internet gateway

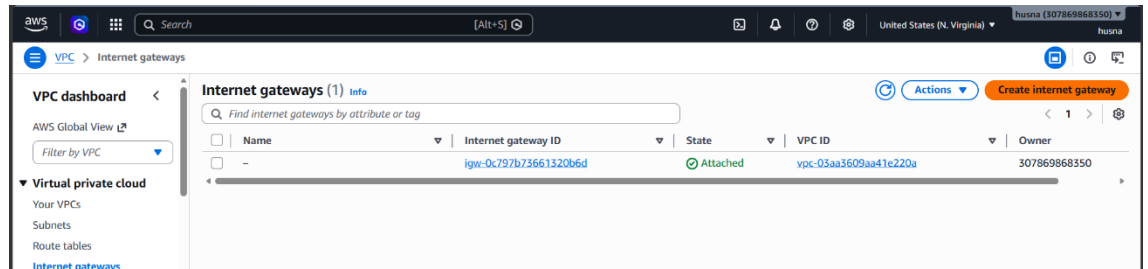


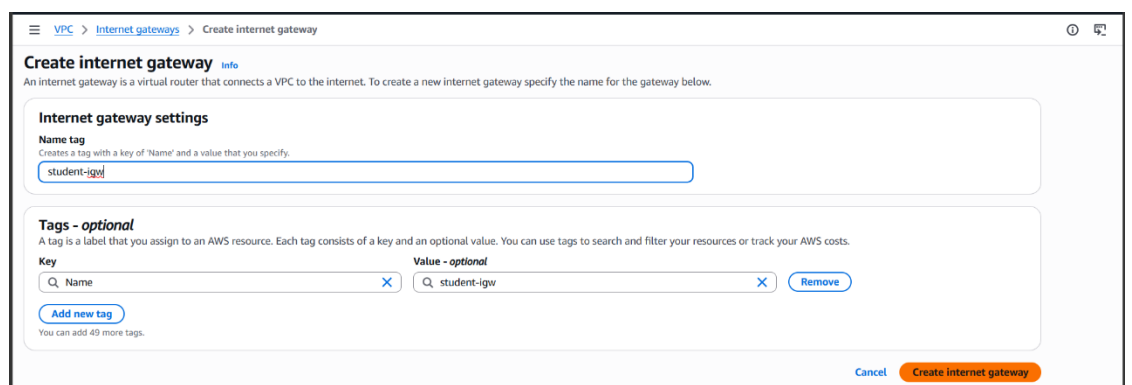
Figure 12: Internet Gateway dashboard before creating a new gateway.

- Enter:

Parameter	Value
Name tag	student-igw

Table 3: Internet Gateway configuration details.

- Click Create internet gateway



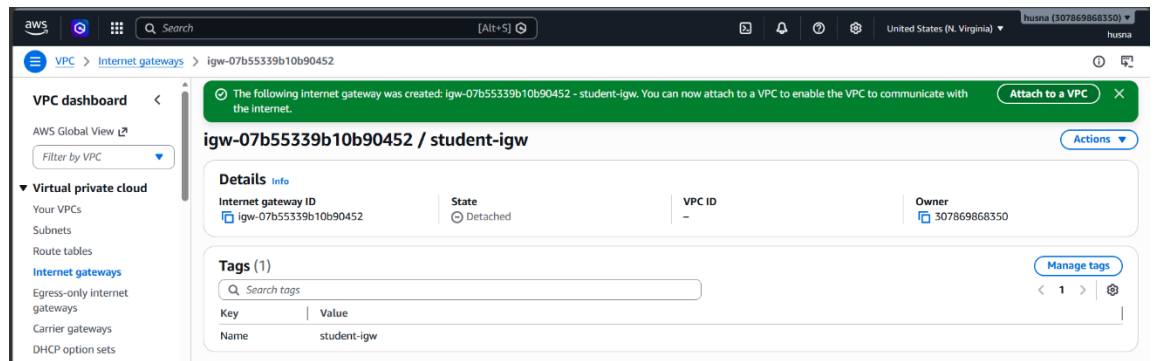


Figure 13: The successful creation of the Internet Gateway.

- Select the created Internet gateway
- Click Actions
- Click Attach to VPC
- Select student-vpc
- Click Attach internet gateway

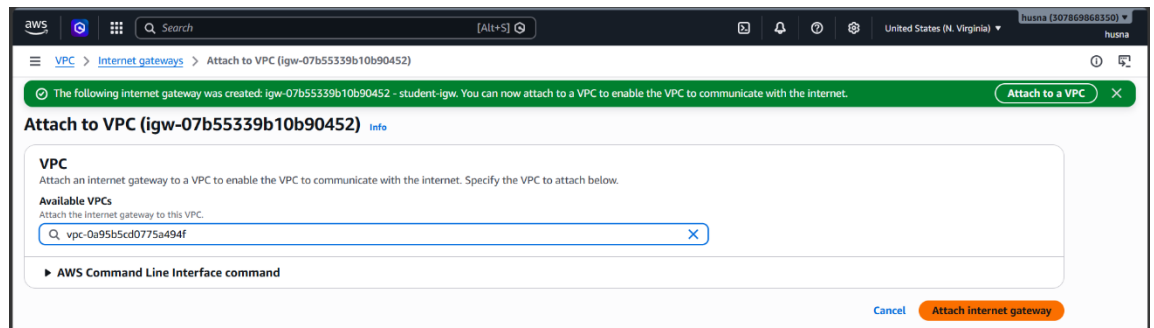


Figure 14: The process of attaching the Internet Gateway to the Student VPC.

- Outcome:

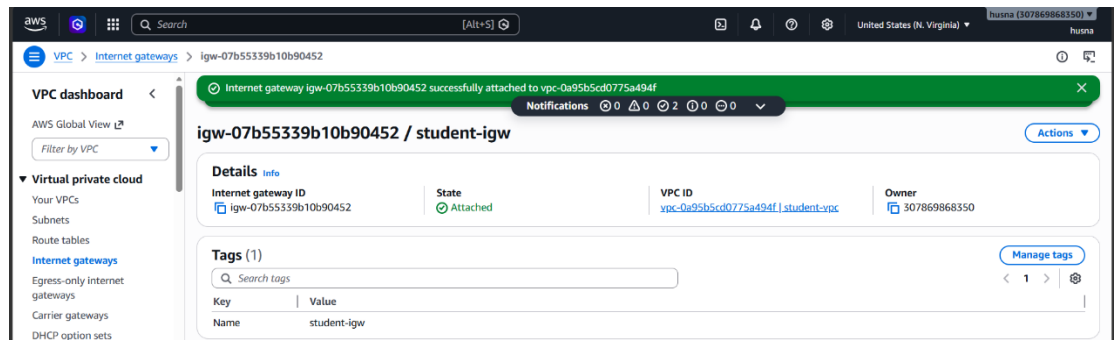


Figure 15: The successfully attached Internet Gateway.

2.5. Create Route Table

- In the left menu, click Route tables
- Click Create route table

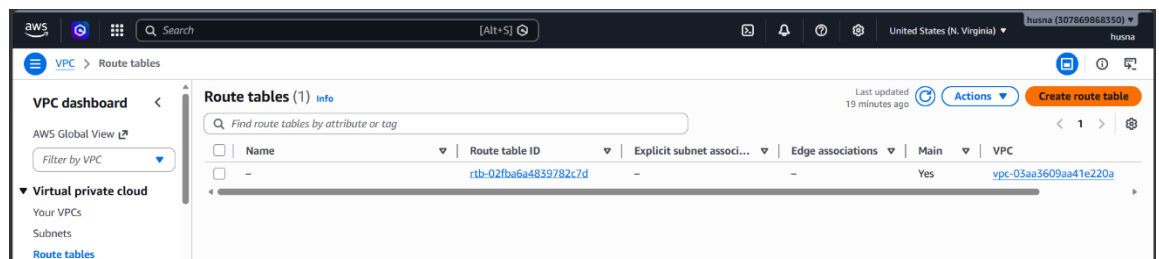


Figure 16: Route Table dashboard used to create a new route table.

- Enter:

Parameter	Value
Name	student-route-table
VPC	student-vpc

Table 4: The route table configuration parameters.

- Click Create route table

Create route table [Info](#)

A route table specifies how packets are forwarded between the subnets within your VPC, the internet, and your VPN connection.

Route table settings

Name - optional
Create a tag with a key of 'Name' and a value that you specify.
student-route-table

VPC
The VPC to use for this route table.
vpc-0a95b5cd0775a494f (student-vpc)

Tags
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key
Q Name

Value - optional
Q student-route-table [Remove](#)

[Add new tag](#)
You can add 49 more tags.

[Cancel](#) [Create route table](#)

Figure 17: The route table creation settings.

- Outcome:

Route table rtb-011d53550b11732d1 | student-route-table was created successfully.

rtb-011d53550b11732d1 / student-route-table [Actions](#)

Details [Info](#)

Route table ID: rtb-011d53550b11732d1
VPC: vpc-0a95b5cd0775a494f | student-vpc
Main: No
Owner ID: 307869868350
Explicit subnet associations: -
Edge associations: -

Routes [Subnet associations](#) [Edge associations](#) [Route propagation](#) [Tags](#)

Routes (1) [Filter routes](#) [Both](#) [Edit routes](#)

Destination	Target	Status	Propagated	Route Origin
10.0.0.0/16	local	Active	No	Create Route Table

Figure 18: The successfully created route table.

2.6. Security Group Verification

- Open student-route-table
- Go to the Routes tab
- Click Edit routes

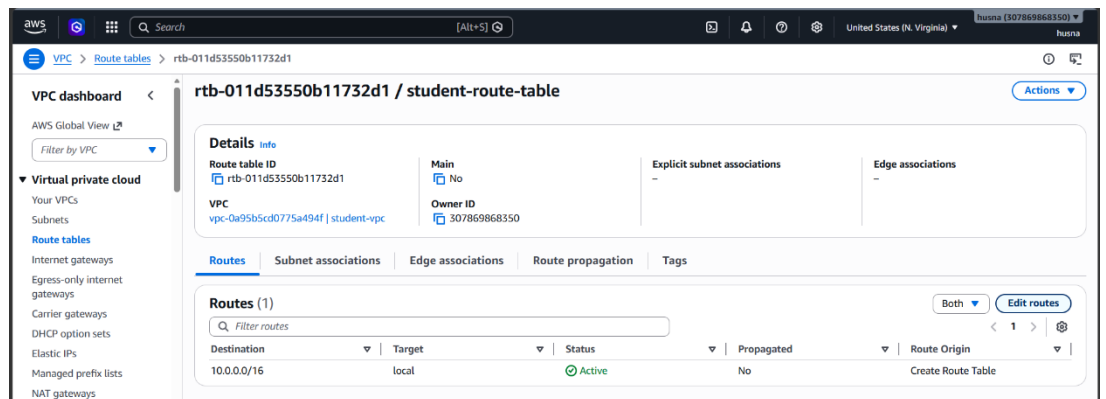


Figure 19: The Route Table interface before adding internet routing.

- Click Add route
- Enter:

Destination	Target
0.0.0.0/0	student-igw

Table 5: The routing rule used to direct internet traffic through the Internet Gateway.

- Click Save changes

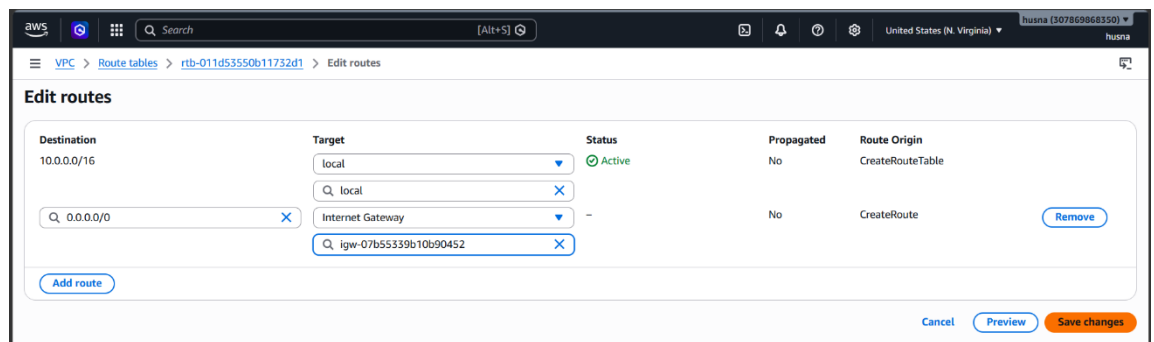


Figure 20: The route configuration process.

- Outcome:

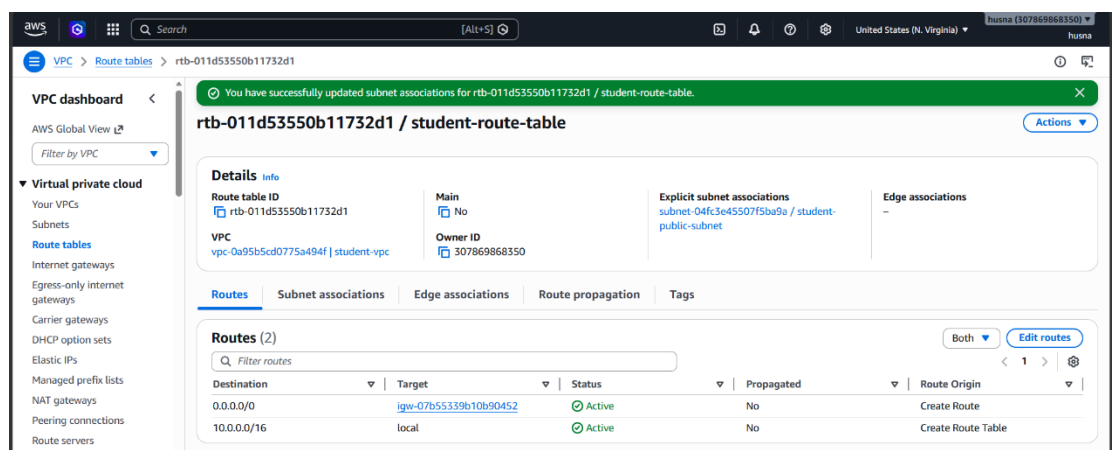


Figure 21: The successful routing configuration for internet access.

2.7. Create Security Group

- In the left menu, click Security Groups
- Click Create security group

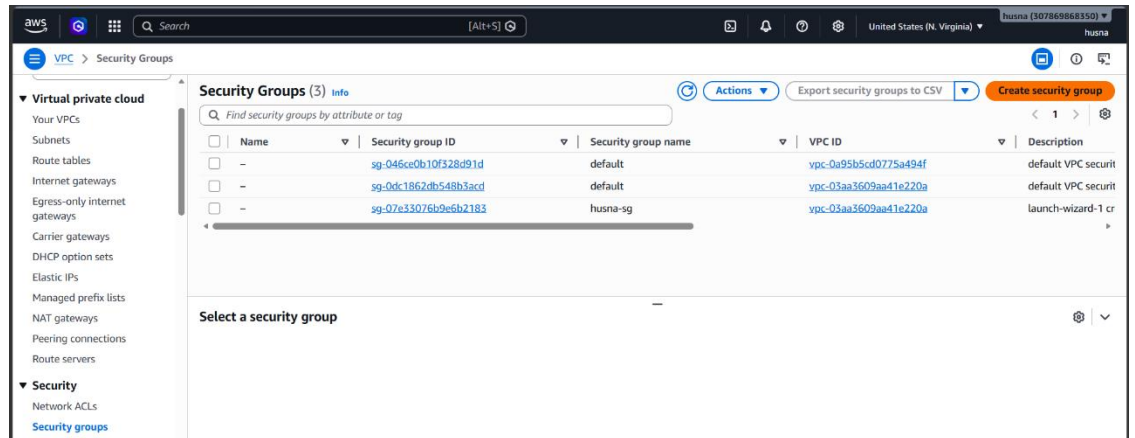


Figure 22: The Security Group dashboard.

- Enter:

Parameter	Value
Security group name	student-sg
Description	Student Registration Security Group
VPC	student-vpc

Table 6: Security Group configuration details.

The screenshot shows the AWS Management Console interface for creating a new security group. The breadcrumb navigation at the top indicates the path: VPC > Security Groups > Create security group. The main heading is 'Create security group' with an 'Info' link. Below this, a note states: 'A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. To create a new security group, complete the fields below.'

The 'Basic details' section contains the following fields:

- Security group name:** A text input field containing 'student-sg'. Below the field, a note says 'Name cannot be edited after creation.'
- Description:** A text input field containing 'Student Registration Security Group'.
- VPC:** A dropdown menu showing 'vpc-0a95b5cd0775a494f (student-vpc)' with a downward arrow.

Figure 23: The Security Group creation settings.

- Scroll to Inbound rules
- Add these rules:

Type	Port	Source	Purpose
SSH	22	0.0.0.0/0	Remote EC2 access
HTTP	80	0.0.0.0/0	Website access
HTTPS	443	0.0.0.0/0	Secure website access
MySQL/Aurora	3306	0.0.0.0/0	RDS database connection

Table 7: The inbound security rules configured for the Student Registration System.

- Leave Outbound rules as default

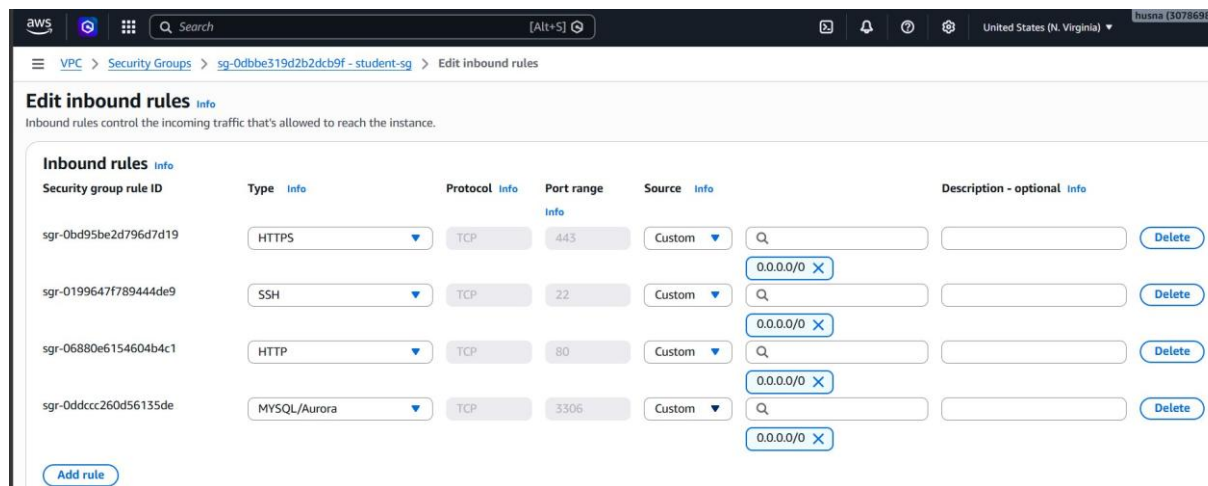


Figure 24: The inbound rule configuration for the Security Group.

- Click Create security group

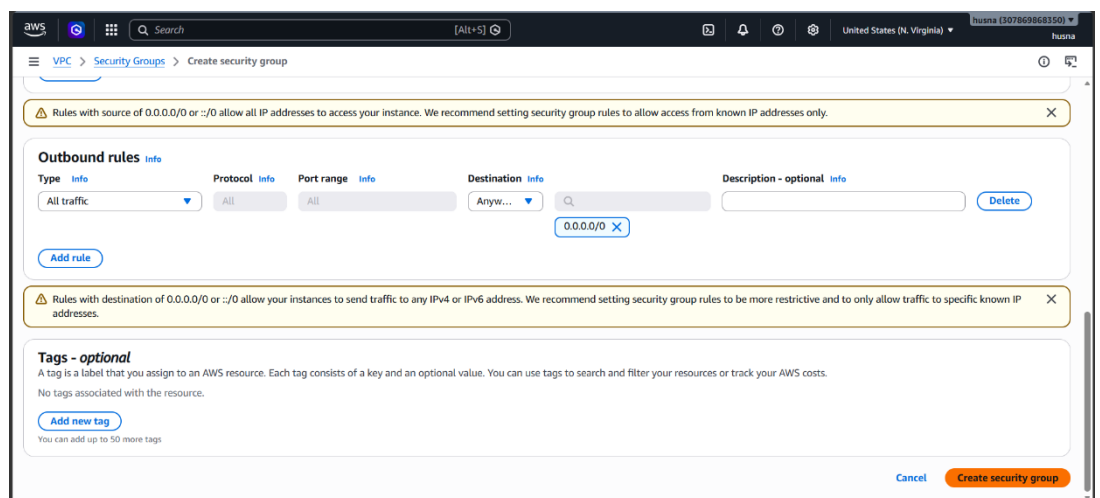


Figure 25: The final Security Group configuration before creation.

- Outcome:

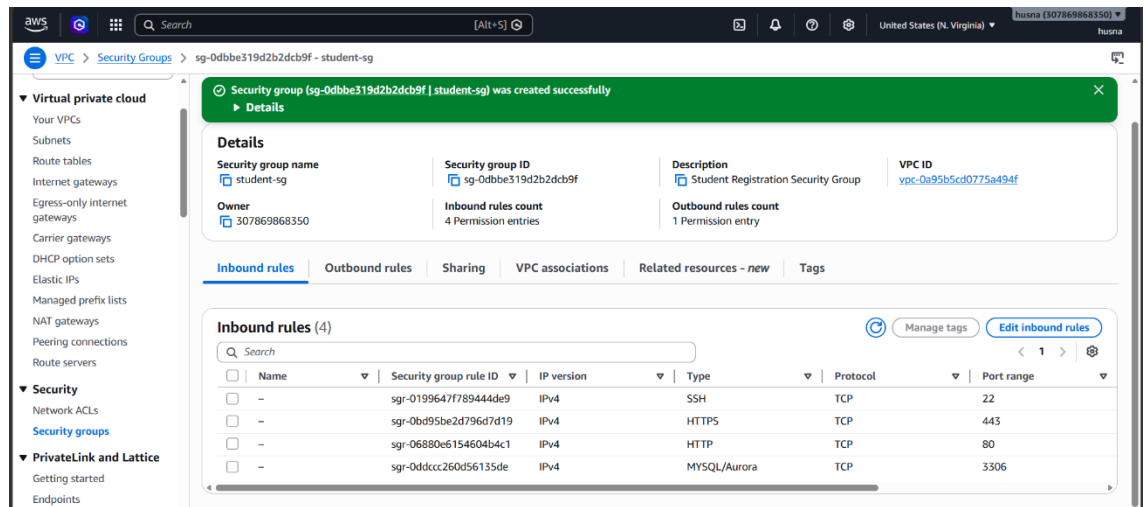


Figure 26: The successfully created Security Group with all inbound rules applied.

2.8. Verification

- Check that these are created:

Component	Expected Result
VPC	student-vpc available
Subnet	student-public-subnet available
Internet Gateway	student-igw attached
Route Table	0.0.0.0/0 points to Internet Gateway
Security Group	SSH, HTTP, HTTPS, MySQL rules added

Table 8: The verification checklist used to validate the networking infrastructure.

3. Web Hosting Implementation using Amazon EC2

As the team member responsible for web hosting and server deployment, Amazon Elastic Compute Cloud (Amazon EC2) was implemented to provide a reliable and scalable cloud computing environment for the Student Registration System. The primary purpose of this component was to host the web application and make it accessible to users through the internet. By utilizing Amazon EC2, the system benefits from flexible computing resources, remote server management and high availability without requiring physical server infrastructure.

For this project, an Amazon EC2 instance was created using the Amazon Linux operating system and the Free Tier eligible t2.micro instance type. The EC2 instance was configured to act as the main web server for the Student Registration System. During the deployment process, a key pair was generated to enable secure remote access, while appropriate network and security configurations were applied to ensure reliable communication between the server and other AWS resources.

To support web hosting functionality, an Apache HTTP Server was installed and configured on the EC2 instance. The web application files, including the Student Registration System interface, were uploaded and deployed to the server. Once the deployment process was completed, the website became accessible through the public IPv4 address assigned to the EC2 instance. This enabled users to access the system through a standard web browser from any location with internet connectivity.

In addition, security measures were implemented through AWS Security Groups to control inbound and outbound network traffic. Specific rules were configured to allow Secure Shell (SSH) access for server administration and Hypertext Transfer Protocol (HTTP) access for website communication. These configurations ensured that the web application remained accessible while maintaining a secure cloud environment.

Several testing procedures were conducted to verify the functionality of the deployed server. These included validating the EC2 instance status, testing remote SSH connectivity, confirming Apache web server operation, and accessing the Student Registration System through its public IP address. The successful completion of these tests demonstrated that the server environment was functioning correctly and capable of supporting the application requirements.

Overall, the implementation of Amazon EC2 significantly enhanced the project by providing a secure, scalable, and cloud-based hosting platform for the Student Registration System. The web hosting component plays a vital role in ensuring continuous application availability, efficient resource utilization, and seamless integration with other AWS services utilized throughout the

project.

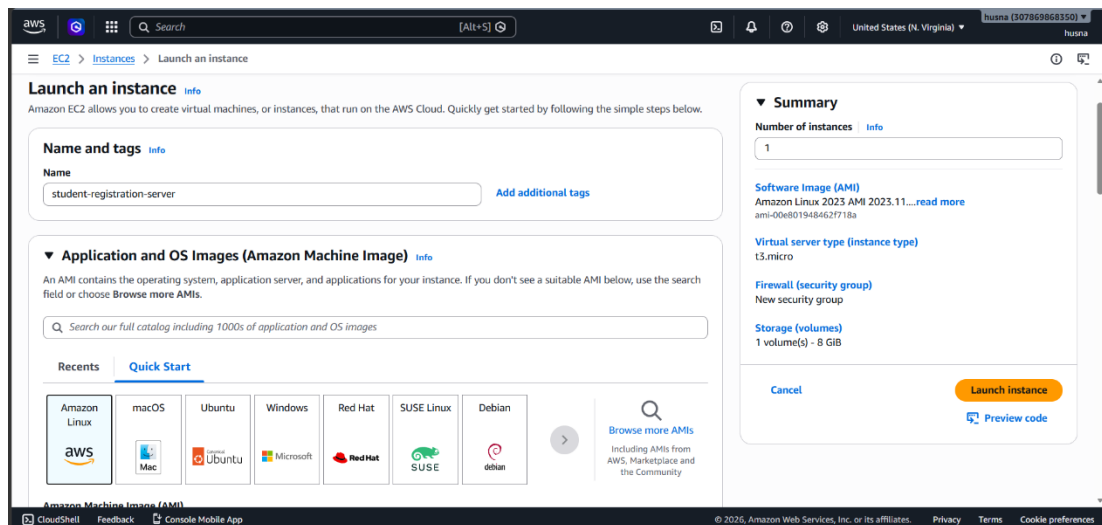
3.1. Creating an EC2 Instance

The first stage involved launching an Amazon EC2 instance through the AWS Management Console.

The following configurations were selected during instance creation:

- Instance Name: student-registration-server
- Amazon Machine Image (AMI): Amazon Linux 2023
- Instance Type: t2.micro (Free Tier Eligible)
- Key Pair: student-key.pem
- Network Settings: Student VPC
- Security Group: student-sg

After reviewing all configurations, the EC2 instance was launched successfully.



Instance type [Info](#) [Get advice](#)

Instance type

t3.micro Free tier eligible

Family: t3 2 vCPU 1 GiB Memory Current generation: true

On-Demand Linux base pricing: 0.0104 USD per Hour

On-Demand Windows base pricing: 0.0196 USD per Hour

On-Demand Ubuntu Pro base pricing: 0.0139 USD per Hour

On-Demand SUSE base pricing: 0.0104 USD per Hour On-Demand RHEL base pricing: 0.0392 USD per Hour

[Additional costs apply for AMIs with pre-installed software](#)

☐ All generations [Compare instance types](#)

Key pair (login) [Info](#)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - *required*

student-key [Create new key pair](#)

Network settings [Info](#) [Edit](#)

Network [Info](#)

vpc-03aa3609a41e220a

Summary

Number of instances [Info](#)

1

Software Image (AMI)

Amazon Linux 2023 AMI 2023.11...[read more](#)

ami-00e80194b4627718a

Virtual server type (instance type)

t3.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

[Cancel](#) [Launch instance](#) [Preview code](#)

Network settings [Info](#)

VPC - *required* [Info](#)

vpc-0a95b5cd0775a494f (student-vpc) [Create new VPC](#)

10.0.0.0/16

Subnet [Info](#)

subnet-04fc3e45507f5ba9a [Create new subnet](#)

VPC: vpc-0a95b5cd0775a494f Owner: 307869868350 Availability Zone: us-east-1a (use 1-az1)

Zone type: Availability Zone IP addresses available: 251 CIDR: 10.0.0.0/24

Auto-assign public IP [Info](#)

☒ Enable

Firewall (security groups) [Info](#)

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☐ Create security group ☒ Select existing security group

Common security groups [Info](#)

Select security groups

student-sg sg-0dbbe319d2b2dcb9f [X](#)

VPC: vpc-0a95b5cd0775a494f

Security groups that you add or remove here will be added to or removed from all your network interfaces.

[Advanced network configuration](#)

Summary

Number of instances [Info](#)

1

Software Image (AMI)

Amazon Linux 2023 AMI 2023.11...[read more](#)

ami-00e80194b4627718a

Virtual server type (instance type)

t3.micro

Firewall (security group)

student-sg

Storage (volumes)

1 volume(s) - 8 GiB

[Cancel](#) [Launch instance](#) [Preview code](#)

Figure 27: The Amazon EC2 instance configuration used to deploy the Student Registration System web server.

3.2. Configuring Security Group Rules

To enable public access to the web application, inbound security rules were configured.

The following ports were allowed:

Port	Protocol	Purpose
22	SSH	Remote server access
80	HTTP	Website Access
443	HTTPS	Secure website access

Table 9: The inbound ports configured to allow SSH, HTTP, and HTTPS access to the EC2 instance.

These rules ensure users can access the web application while allowing administrators to manage the server remotely.

3.3. Connecting to the RC2 Instance

After the EC2 instance entered the running state, a connection was established using SSH. The following command was executed in Command Prompt:

```
ssh -i student-key.pem ec2-user@Public-IP-Address
```

A successful login verified that the server was accessible and ready for deployment

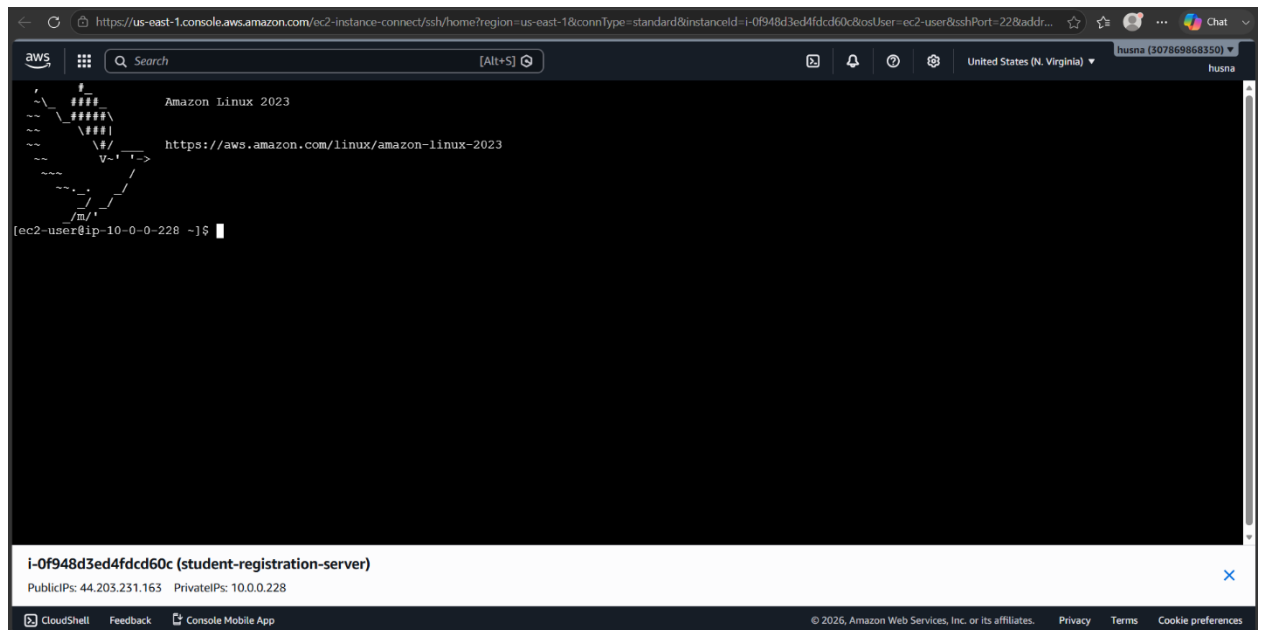
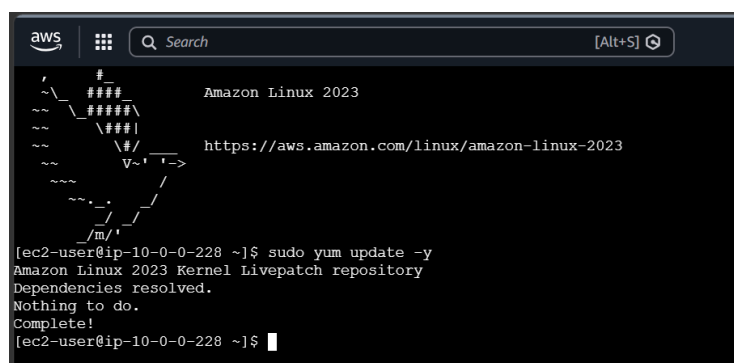


Figure 28: The successful SSH connection to the EC2 instance.

3.4. Installing Apache Web Server

Apache HTTP Server was installed to host the Student Registration System website. The following commands were executed:

- `sudo yum update -y`
- `sudo yum install httpd -y`



```
[ec2-user@ip-10-0-0-228 ~]$ sudo yum install httpd -y
Last metadata expiration check: 0:00:45 ago on Thu Jun 4 15:13:04 2026.
Dependencies resolved.
```

Package	Architecture	Version	Repository	Size
Installing:				
httpd	x86_64	2.4.67-1.amzn2023.0.1	amazonlinux	46 k
Installing dependencies:				
apr	x86_64	1.7.5-1.amzn2023.0.4	amazonlinux	129 k
apr-util	x86_64	1.6.3-1.amzn2023.0.2	amazonlinux	97 k
apr-util-ldap	x86_64	1.6.3-1.amzn2023.0.2	amazonlinux	13 k
generic-logos-httpd	noarch	18.0.0-12.amzn2023.0.3	amazonlinux	19 k
httpd-core	x86_64	2.4.67-1.amzn2023.0.1	amazonlinux	1.4 M
httpd-filesystem	noarch	2.4.67-1.amzn2023.0.1	amazonlinux	12 k
httpd-tools	x86_64	2.4.67-1.amzn2023.0.1	amazonlinux	80 k
libbrotli	x86_64	1.0.9-4.amzn2023.0.2	amazonlinux	315 k
mailcap	noarch	2.1.49-3.amzn2023.0.3	amazonlinux	33 k
Installing weak dependencies:				
apr-util-openssl	x86_64	1.6.3-1.amzn2023.0.2	amazonlinux	15 k
mod_http2	x86_64	2.0.39-1.amzn2023.0.1	amazonlinux	167 k
mod_lua	x86_64	2.4.67-1.amzn2023.0.1	amazonlinux	59 k

```
Transaction Summary
Install 13 Packages
Total download size: 2.4 M
```

Figure 29: The installation process of the Apache HTTP web server on Amazon EC2.

After installation, the Apache service was started and enabled automatically during system startup.

- `sudo systemctl start httpd`
- `sudo systemctl enable httpd`

```
[ec2-user@ip-10-0-0-228 ~]$ sudo systemctl start httpd
[ec2-user@ip-10-0-0-228 ~]$
```

```
[ec2-user@ip-10-0-0-228 ~]$ sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[ec2-user@ip-10-0-0-228 ~]$
```

Figure 30: The commands used to start and enable the Apache web server service.

The status of the service was checked to ensure successful installation.

- `sudo systemctl status httpd`

```

[ec2-user@ip-10-0-0-228 ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-06-04 15:14:48 UTC; 1min 8s ago
     Docs: man:httpd.service(8)
  Main PID: 26676 (httpd)
   Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
    Tasks: 177 (limit: 1014)
   Memory: 13.5M
      CPU: 129ms
   CGroup: /system.slice/httpd.service
           └─26676 /usr/sbin/httpd -DFOREGROUND
             └─26677 /usr/sbin/httpd -DFOREGROUND
               └─26678 /usr/sbin/httpd -DFOREGROUND
                 └─26679 /usr/sbin/httpd -DFOREGROUND
                   └─26680 /usr/sbin/httpd -DFOREGROUND

Jun 04 15:14:48 ip-10-0-0-228.ec2.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
Jun 04 15:14:48 ip-10-0-0-228.ec2.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
Jun 04 15:14:48 ip-10-0-0-228.ec2.internal httpd[26676]: Server configured, listening on: port 80
[ec2-user@ip-10-0-0-228 ~]$

```

Figure 31: The Apache service status, confirming successful installation and operation.

3.5. Deploying the Website

The HTML files for the Student Registration System were uploaded to the Apache web directory. The default webpage was replaced with the project files using the following commands:

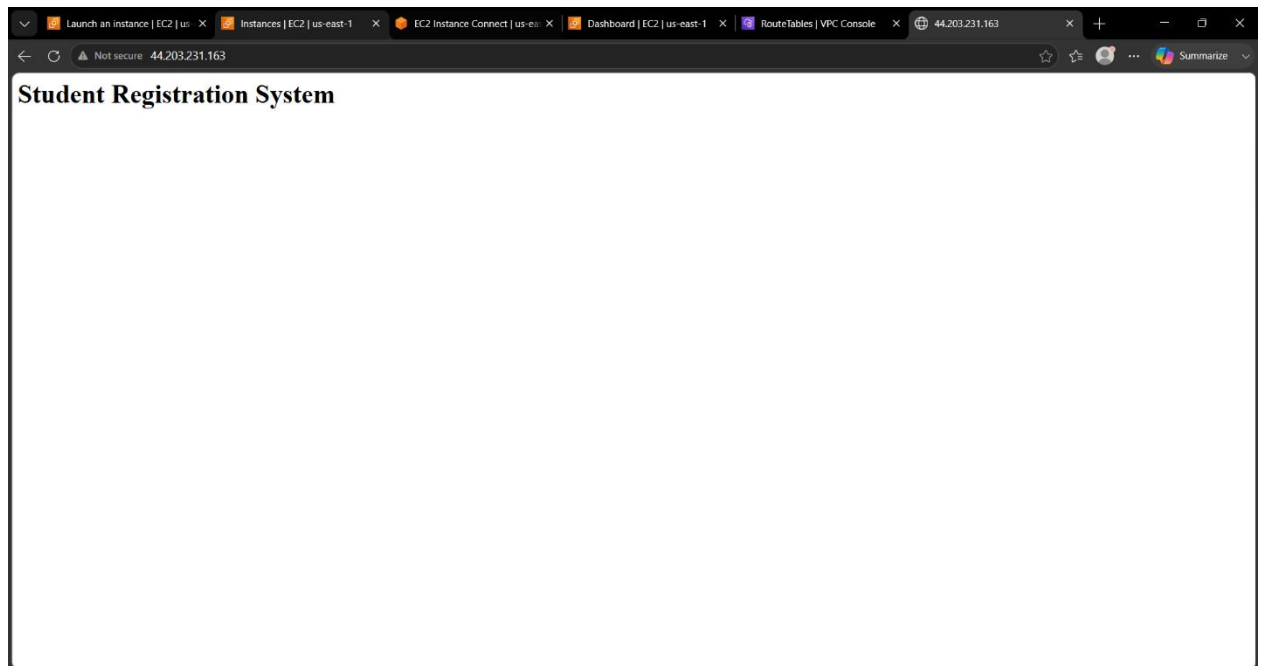
- `cd /var/www/html`
- `sudo rm index.html`
- `sudo nano index.html`

The website source code was then saved and deployed to the server.

```

[ec2-user@ip-10-0-0-228 ~]$ echo "<h1>Student Registration System</h1>" | sudo tee /var/www/html/index.html
<h1>Student Registration System</h1>
[ec2-user@ip-10-0-0-228 ~]$

```

```
aws | Search [Alt+S]
GNU nano 8.3 /var/v
<!DOCTYPE html>
<html>
<head>
  <title>Student Registration System</title>
</head>
<body>

<h1>Welcome to Student Registration System</h1>

<a href="register.html">Register Student</a>
<br><br>

<a href="students.html">View Students</a>

</body>
</html>
```

```
aws | [Search] [Alt+S]
GNU nano 8.3 /var/v
<!DOCTYPE html>
<html>
<head>
  <title>Student Registration System</title>
</head>
<body>

<h1>Welcome to Student Registration System</h1>

<a href="register.html">Register Student</a>
<br><br>

<a href="students.html">View Students</a>

</body>
</html>
```

```
[ec2-user@ip-10-0-0-228 ~]$ sudo nano /var/www/html/index.html
[ec2-user@ip-10-0-0-228 ~]$
```

```
[ec2-user@ip-10-0-0-228 ~]$ sudo nano /var/www/html/register.html
```

```
[ec2-user@ip-10-0-0-228 ~]$ sudo nano /var/www/html/register.html
```

```
aws | [App Icon] | Search [Alt]
GNU nano 8.3
<!DOCTYPE html>
<html>
<head>
<title>Student Registration</title>
</head>
<body>

<h2>Student Registration Form</h2>

<form>

<label>Name:</label><br>
<input type="text"><br><br>

<label>Email:</label><br>
<input type="email"><br><br>

<label>Phone:</label><br>
<input type="text"><br><br>

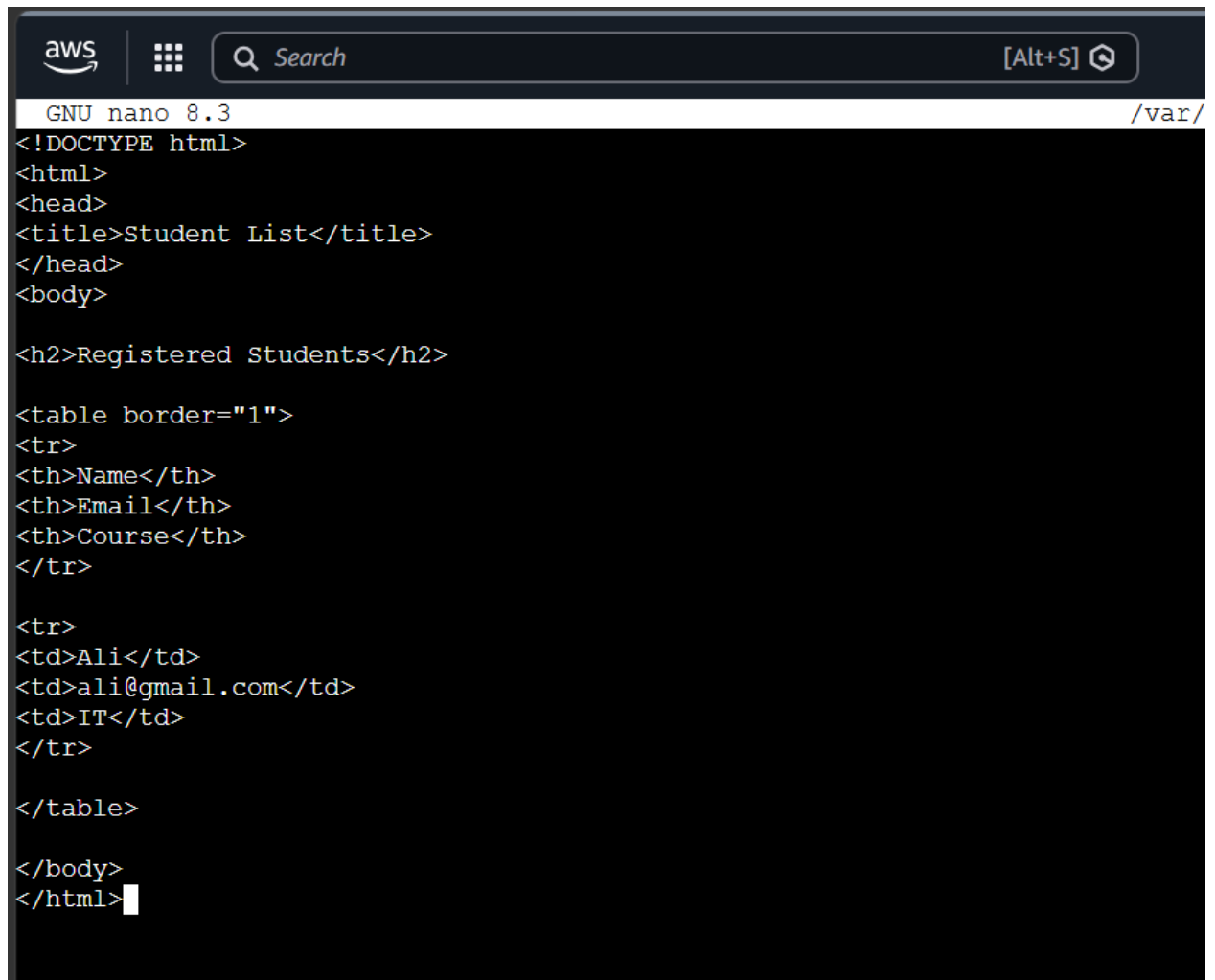
<label>Course:</label><br>
<input type="text"><br><br>

<input type="submit" value="Register">

</form>

</body>

[ec2-user@ip-10-0-0-228 ~]$ sudo nano /var/www/html/students.html
```

A screenshot of an AWS console window. The top bar shows the AWS logo, a search bar with the text "Search", and a keyboard shortcut "[Alt+S]". The main area displays a terminal window titled "GNU nano 8.3" with the file path "/var/". The terminal shows the content of an HTML file being edited. The code includes a DOCTYPE declaration, html, head, and body tags. The title is "Student List". The body contains an h2 tag "Registered Students" and a table with one data row. The table has three columns: Name, Email, and Course. The data row contains "Ali", "ali@gmail.com", and "IT".

```
GNU nano 8.3 /var/
<!DOCTYPE html>
<html>
<head>
<title>Student List</title>
</head>
<body>

<h2>Registered Students</h2>

<table border="1">
<tr>
<th>Name</th>
<th>Email</th>
<th>Course</th>
</tr>

<tr>
<td>Ali</td>
<td>ali@gmail.com</td>
<td>IT</td>
</tr>

</table>

</body>
</html>
```

Figure 32: The deployed Student Registration System web pages stored on the EC2 web server.

3.6. Testing Website Accessibility

The public IPv4 address assigned to the EC2 instance was used to access the website through a browser. (<http://44.203.231.163>)

The Student Registration System homepage was displayed successfully indicating that the deployment process had been completed.

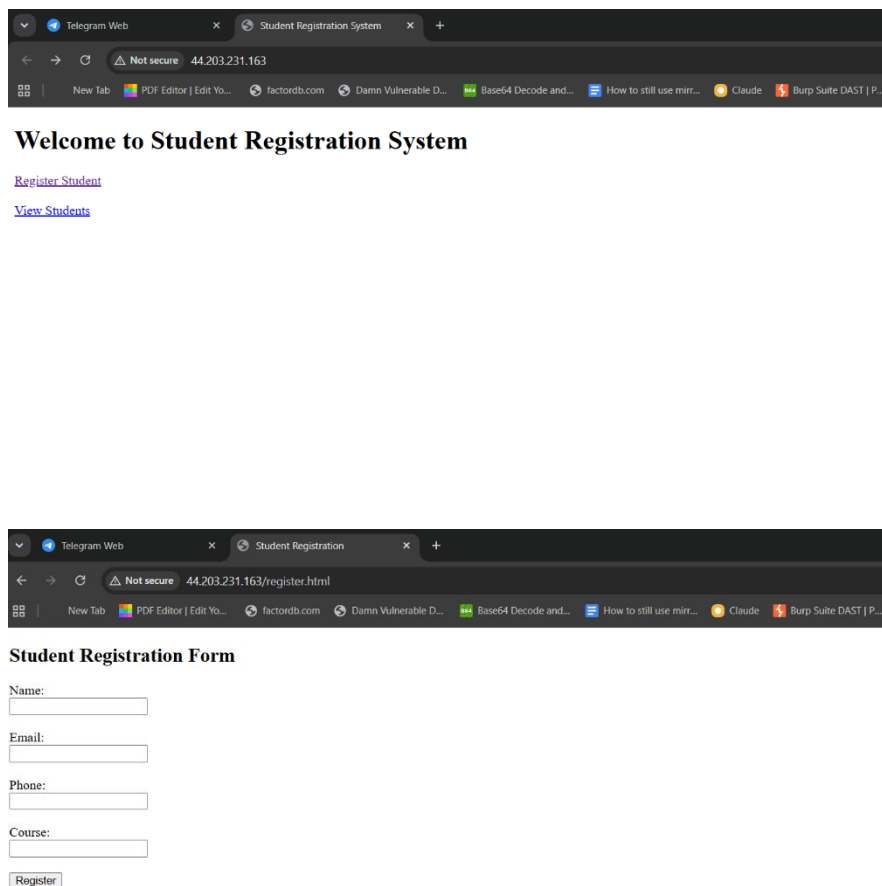


Figure 33: Student Registration System homepage accessed through the EC2 public IP address.

4. Storage Services Implementation using Amazon S3

Amazon Simple Storage Service, also known as Amazon S3, was implemented as the cloud storage service for the Student Registration System. Amazon S3 was selected because it provides secure, scalable, and reliable object storage for storing files such as student photos and supporting documents.

In this system, Amazon S3 is used to store student-related files separately from the web server and database. This separation improves the overall system structure because Amazon EC2 is used to host the web application, Amazon RDS is used to manage student information, and Amazon S3 is used to store uploaded files. By using a dedicated cloud storage service, the system becomes more organized, scalable, and easier to maintain.

4.1. Accessing Amazon S3

The storage implementation began by accessing Amazon S3 through the AWS Management Console. The S3 service was searched using the AWS search bar, and the Amazon S3 dashboard was opened. The S3 dashboard provides access to bucket creation, file upload, storage configuration, versioning, and object management features.

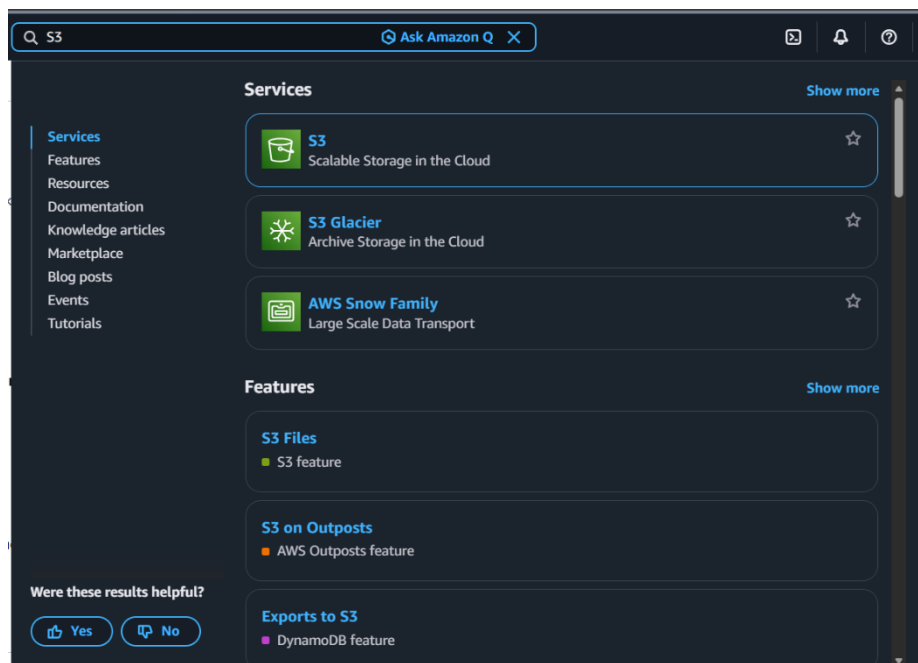
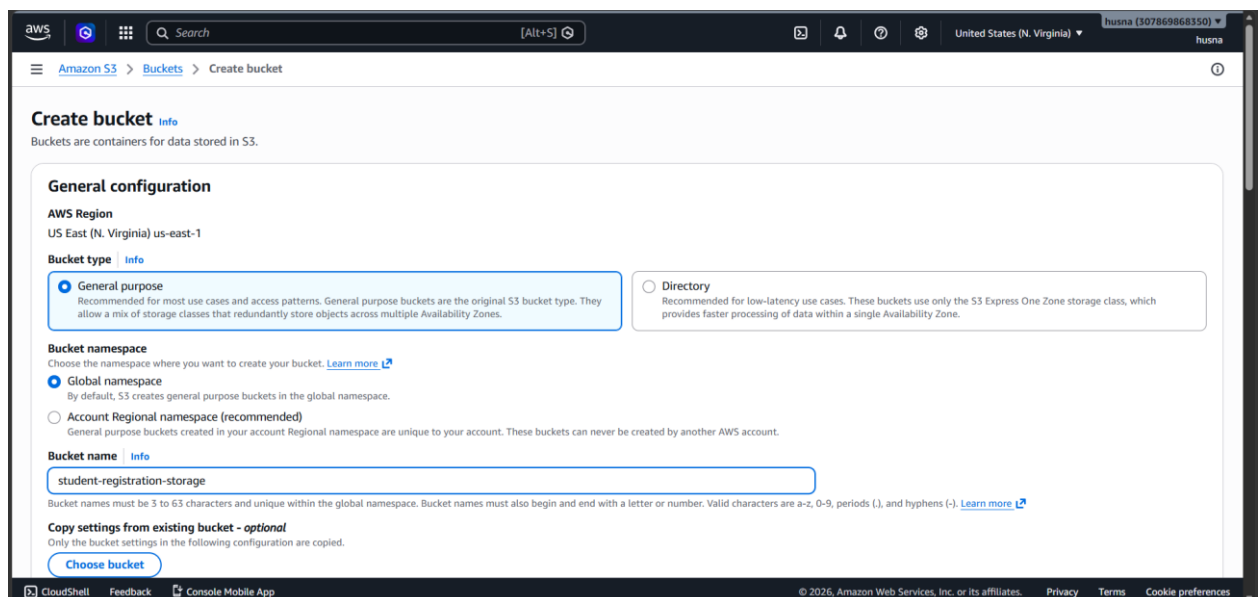
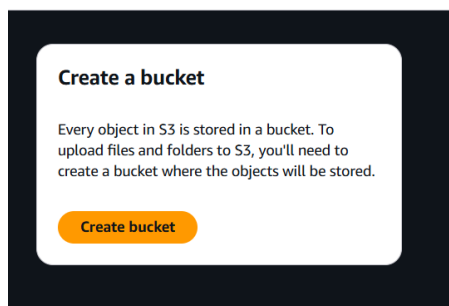


Figure 34: AWS Console search result used to access Amazon S3.

4.2. Creating the S3 Bucket

A new S3 bucket was created for the Student Registration System. An S3 bucket acts as a cloud storage container where files can be uploaded, stored, and managed. For this project, the bucket was created specifically to store student photos and documents related to the registration system.

A unique bucket name was used because S3 bucket names must be globally unique. The bucket was created in the same AWS region as the other project resources to maintain consistency across the cloud environment. Creating a dedicated bucket for the system helps separate file storage from the website and database components.



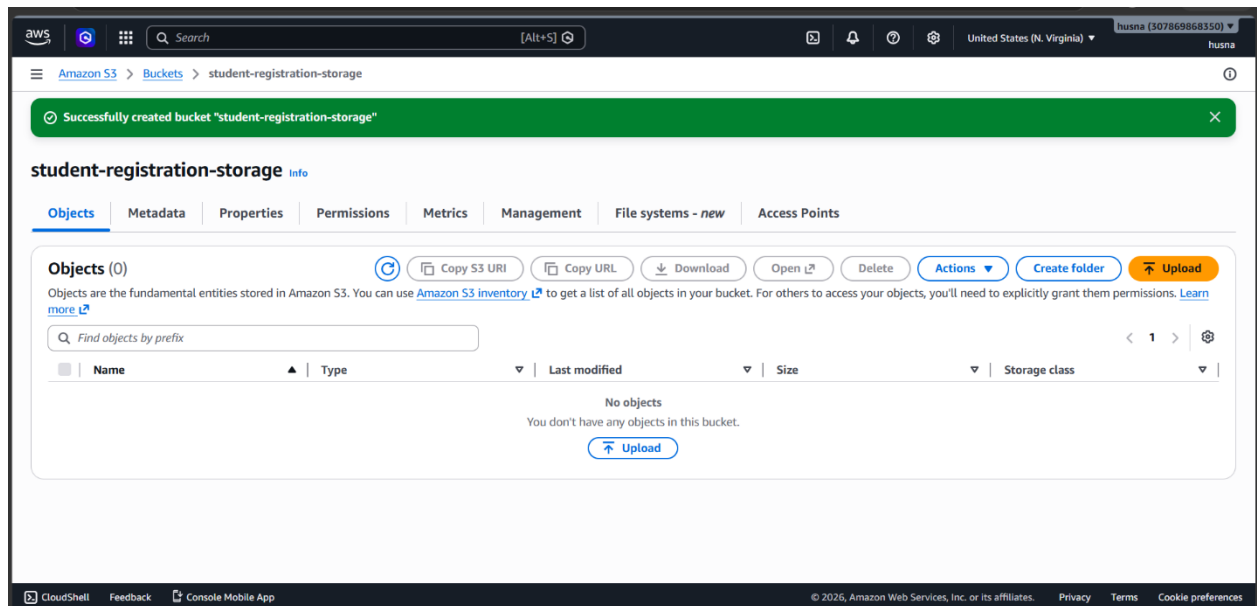


Figure 35: The creation of the S3 bucket used for storing student files and project assets.

4.3. Configuring Bucket Settings

After the bucket was created, the bucket settings were configured. The default object ownership setting was maintained, and Block Public Access was enabled for security purposes. This setting is important because student photos and documents may contain private information and should not be publicly accessible without proper permission.

By blocking public access, files stored in the bucket are protected from unauthorized access through the internet. In a real application, access to the stored files should be controlled through application permissions or secure access policies. Therefore, this configuration supports better security for the Student Registration System.

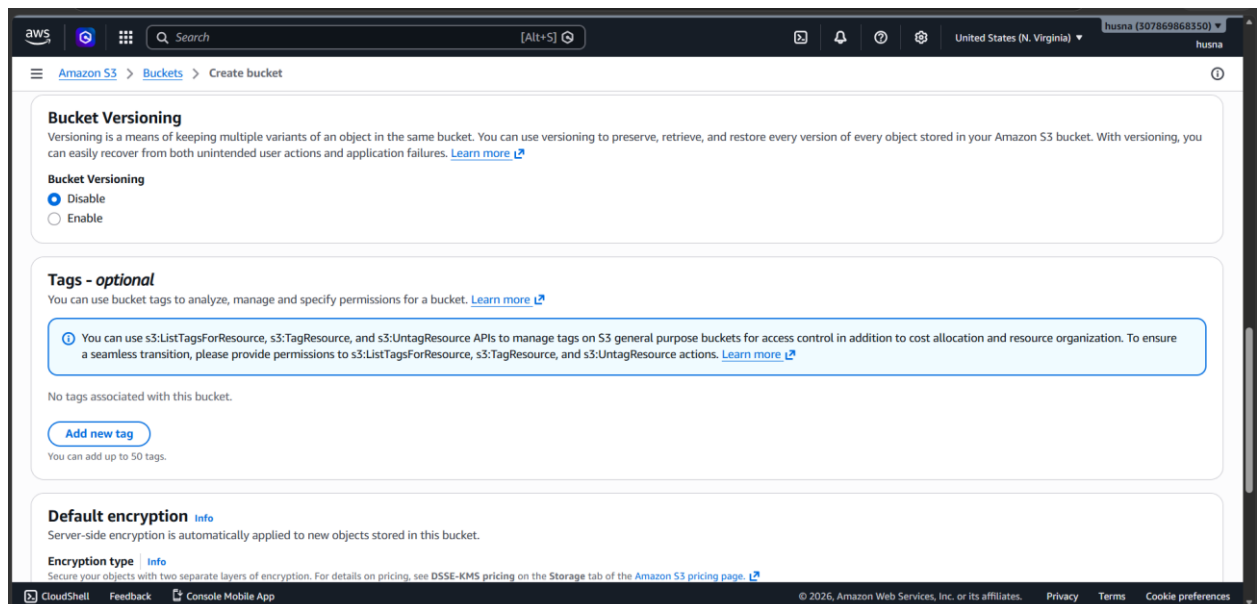
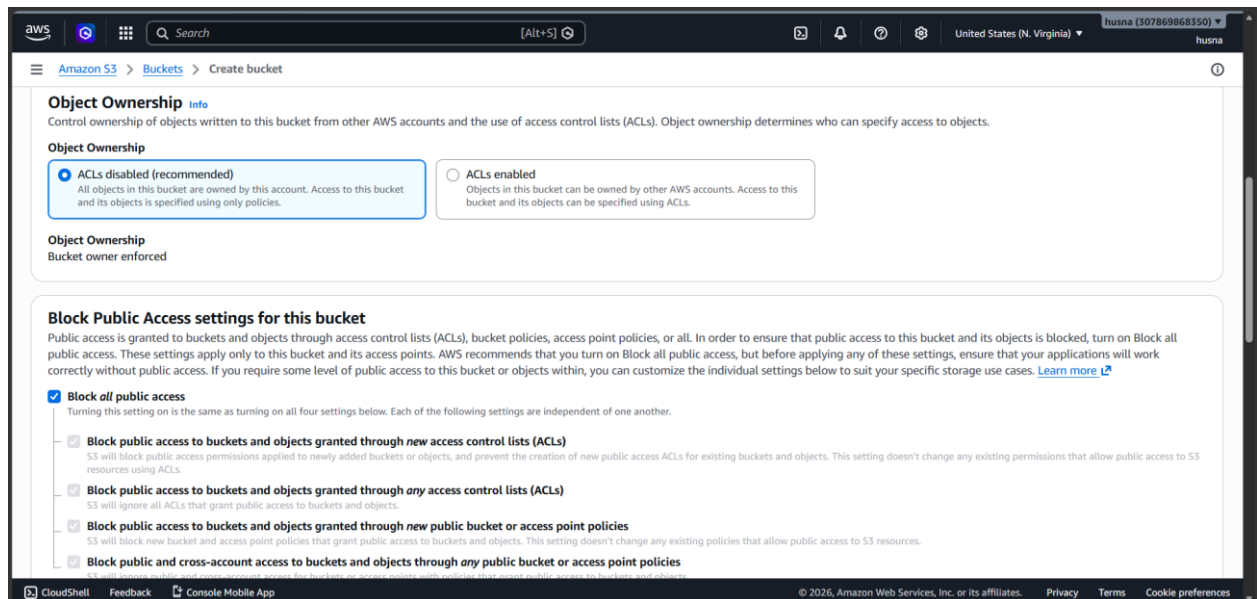


Figure 36: The bucket security settings with Block Public Access enabled.

4.4. Enabling Bucket Versioning

Bucket versioning was enabled to improve file protection and backup management. Versioning allows Amazon S3 to keep multiple versions of the same file. This means that if a file is accidentally replaced, modified, or deleted, the previous version can still be recovered.

For the Student Registration System, versioning is useful because student photos and documents may be updated over time. For example, if a student profile photo is replaced, the older version can still be retained. This feature helps reduce the risk of accidental data loss and improves the reliability of the storage service.

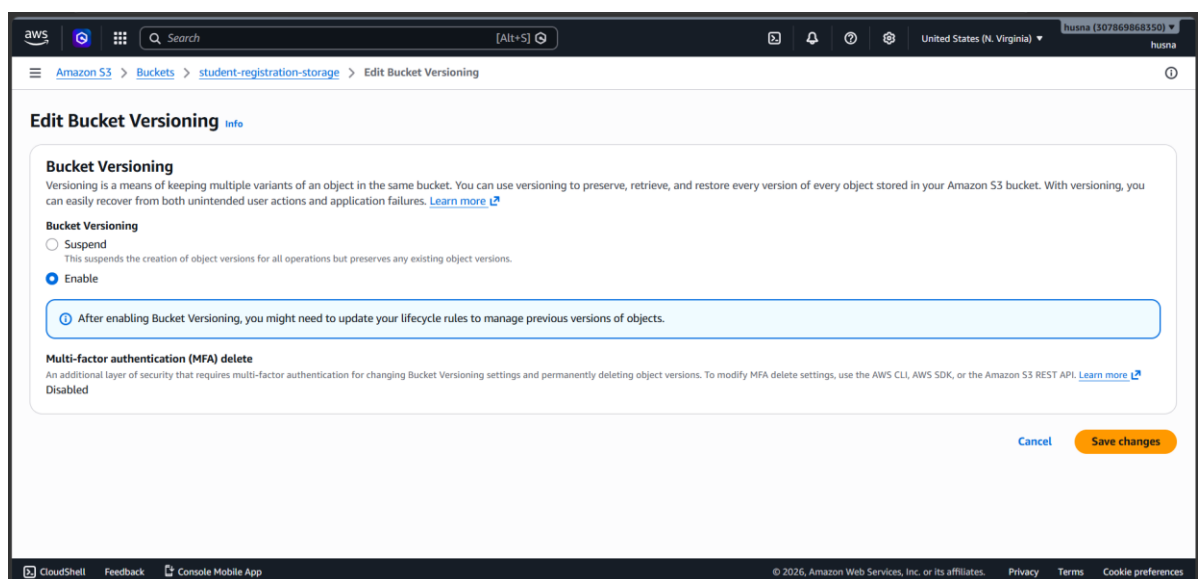


Figure 37: The bucket versioning configuration used for file protection and recovery.

4.5. Creating Folder Structure

A folder named student-photos was created inside the S3 bucket to organize uploaded student images. This folder is used to store student profile photos in a structured location. Although Amazon S3 stores files as objects, folder structures help organize related files and make file management easier.

The use of folder organization is especially useful when the number of uploaded files increases. In the future, additional folders such as documents can also be created to store supporting files such as student certificates, identification documents, or registration forms.

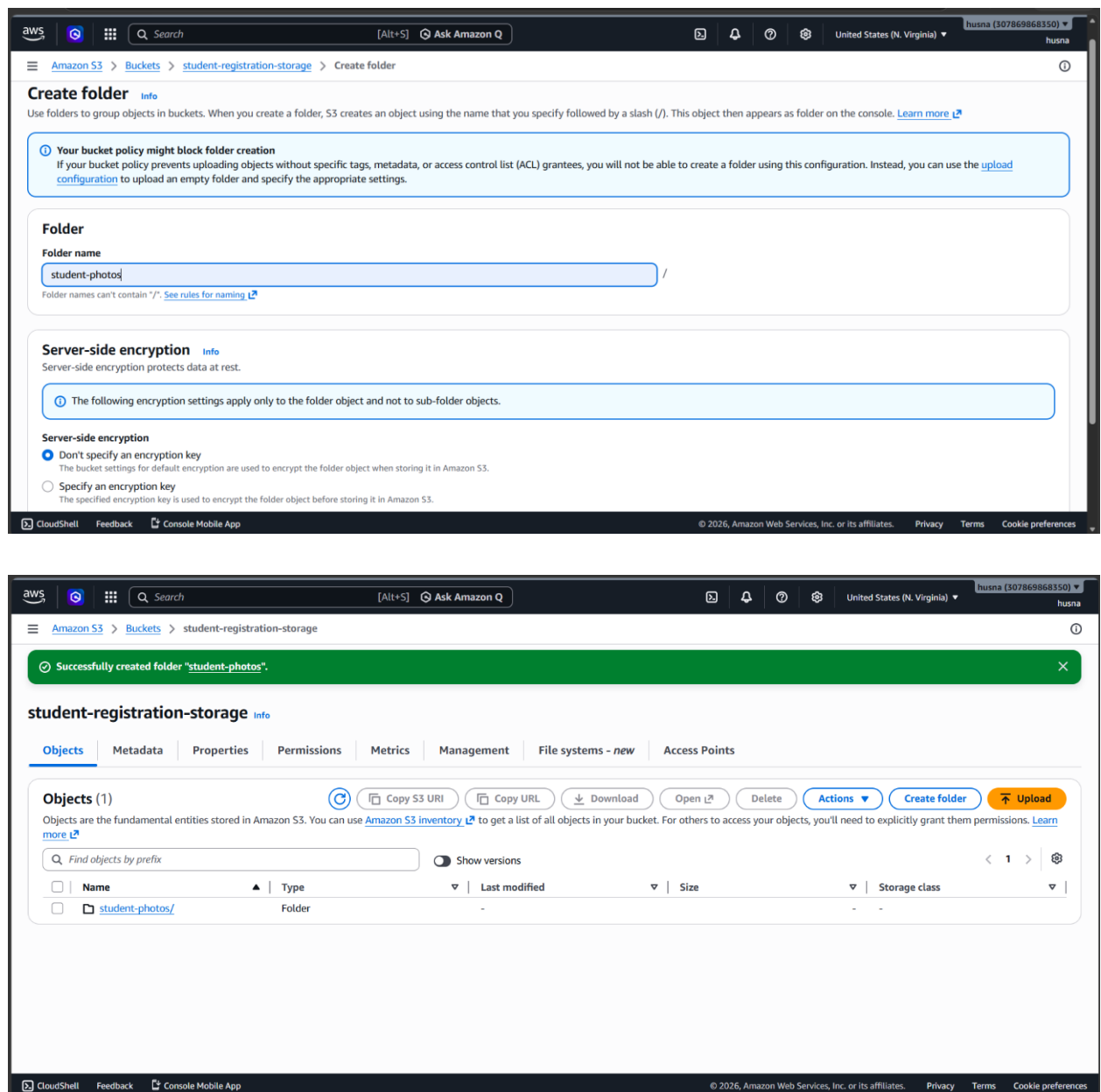


Figure 38: The creation of the student-photos folder within the S3 bucket.

4.6. Uploading Student Files

Sample student photos were uploaded into the student-photos folder. These uploaded files represent student profile images that can be used in the Student Registration System. The successful upload process shows that the S3 bucket is functioning as the cloud storage component of the system.

Each uploaded file is stored as an object in Amazon S3. The uploaded objects can be viewed, managed, and monitored through the S3 console. This demonstrates that the system can store file-based data separately from text-based student information stored in Amazon RDS.

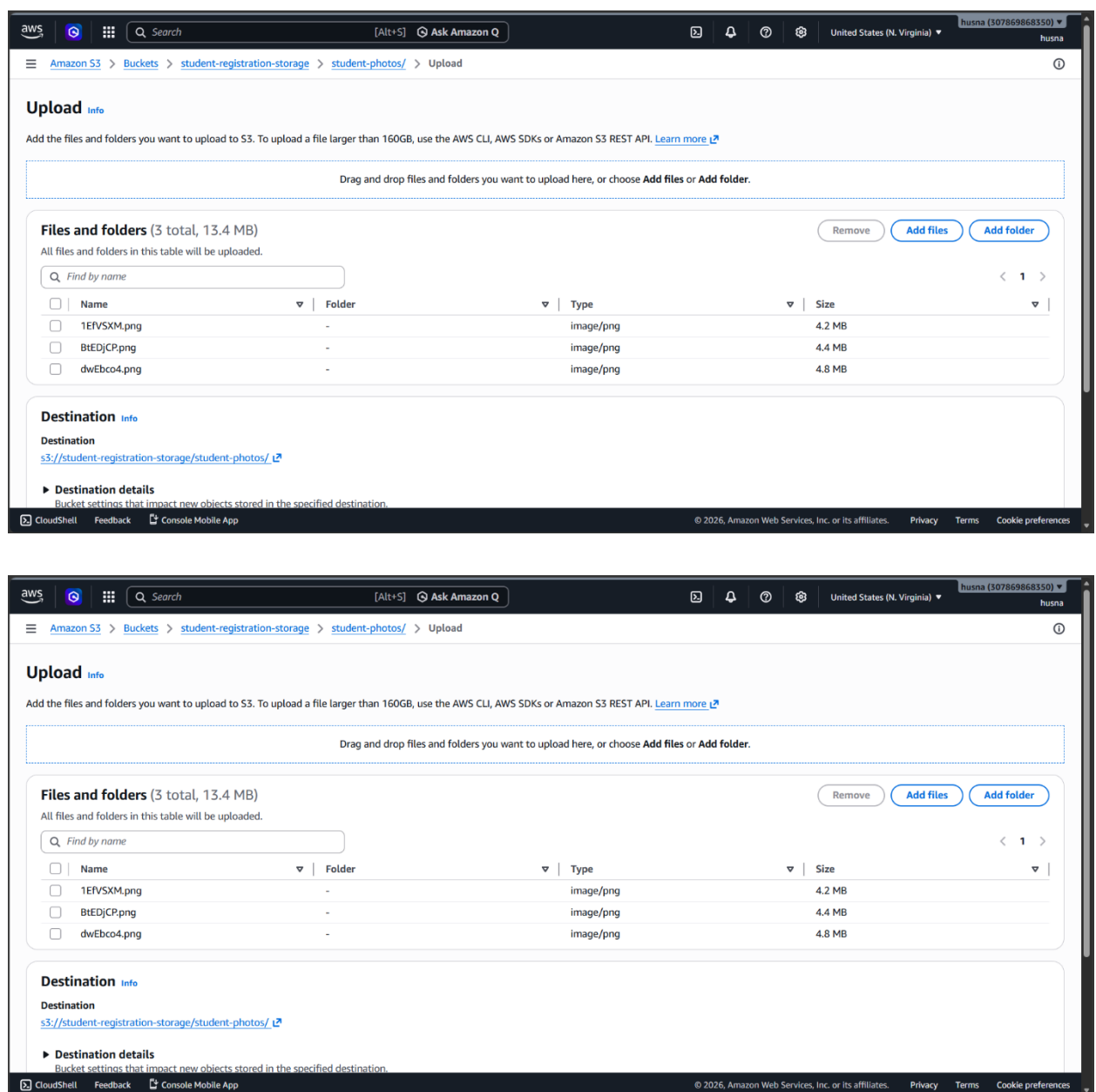


Figure 39: The uploaded student files stored successfully in Amazon S3.

4.7. Verifying Uploaded Objects

After the files were uploaded, the object details were checked to verify that the upload process was successful. The object details page displays important information such as the file name, file size, storage class, last modified date, and object URL.

This verification step confirms that the uploaded files were stored correctly inside the S3 bucket. It also provides evidence that the storage component is working properly and that the files can be managed through Amazon S3.

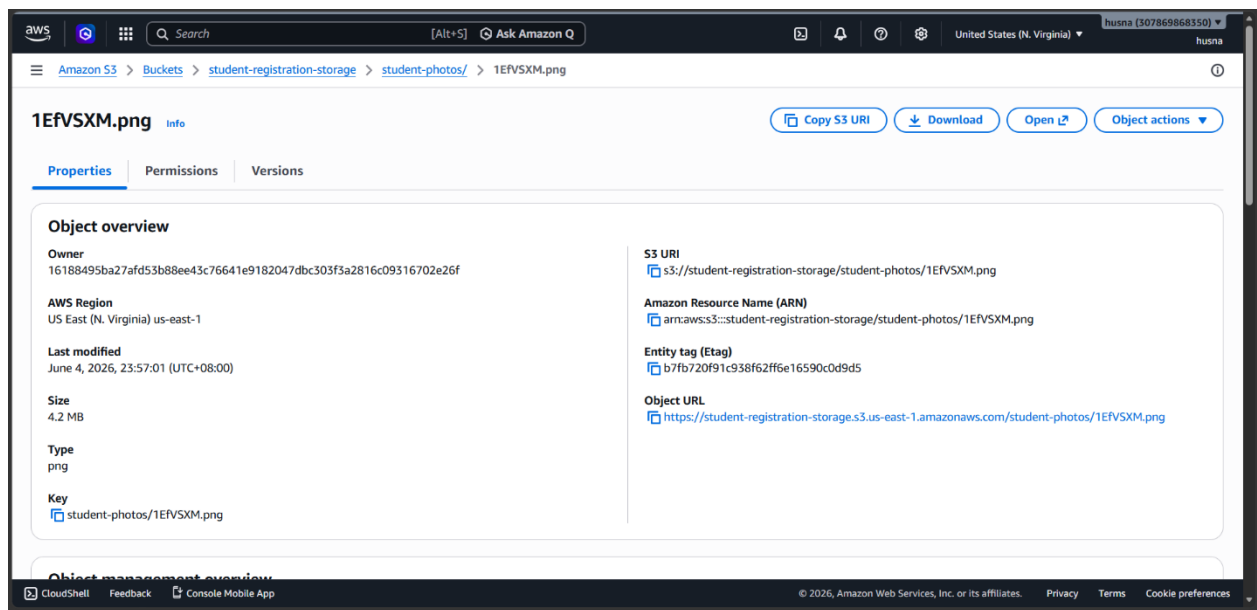


Figure 40: Object Details of Uploaded Student File

4.8. Role of Amazon S3 in the Student Registration System

Amazon S3 plays an important role as the storage component in the Student Registration System. The overall system uses Amazon EC2 to host the website, Amazon RDS to store student information, and Amazon S3 to store student photos and documents.

The storage flow of the system can be summarized as follows:

This structure allows the system to manage files more efficiently. Instead of storing uploaded files directly on the EC2 server, the files are stored in a dedicated cloud storage service. This improves scalability, organization, and maintainability.

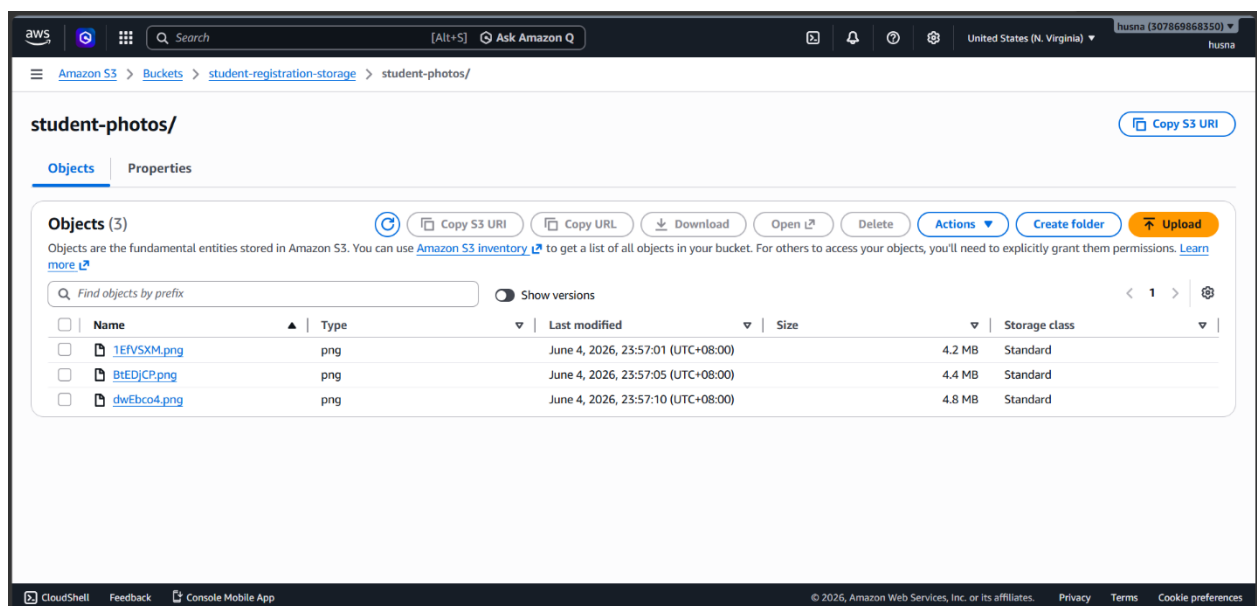


Figure 41: Role of S3 in the Student Registration System

5. Database Services Implementation using Amazon RDS

As the team member responsible for database services, Amazon Relational Database Service (Amazon RDS) was implemented to provide a reliable and scalable cloud database solution for the Student Registration System. The primary purpose of this component was to store, manage, and organize student information in a centralized database environment. By utilizing Amazon RDS, database administration tasks such as infrastructure management, maintenance, and monitoring are simplified, allowing the system to focus on data management and application functionality.

For this project, a MySQL database engine was selected due to its compatibility, ease of use, and widespread adoption in web-based applications. The database instance was created using the Amazon RDS Easy Create configuration, which automatically applies recommended settings and simplifies the deployment process. A database instance named "student-db" was successfully created and configured within the AWS cloud environment.

To ensure secure communication between the application and the database, the RDS instance was connected to the Amazon EC2 server hosting the Student Registration System. AWS automatically configured the required networking and security settings, allowing the web application to communicate with the database through the MySQL default port (3306). This integration enabled the application to access and manage student records efficiently while maintaining a secure cloud architecture.

After the database deployment was completed, a dedicated database named "studentdb" was created to store student-related information. A table named "students" was then designed and implemented to manage student registration records. The table includes important attributes such as student identification number, student name, gender, identity card number, email address, phone number, home address, enrolled course, photo reference, and registration date. This structure allows the system to maintain comprehensive student information while supporting future scalability and data expansion.

Several Structured Query Language (SQL) operations were performed to validate the functionality of the database. These operations included creating the database, creating the student table, inserting sample student records, retrieving stored information, updating existing records, and deleting unwanted records. The successful execution of these operations demonstrated that the database was functioning correctly and capable of supporting the requirements of the Student Registration System.

Overall, the implementation of Amazon RDS significantly enhanced the system by providing a secure, reliable, and cloud-based database solution. The database component plays a critical

role in ensuring data integrity, efficient data management, and seamless integration with other AWS services used throughout the project.

5.1. Accessing Amazon RDS

Amazon RDS was selected as the cloud database service for the Student Registration System. The service was accessed through the AWS Management Console to create and manage the database instance.

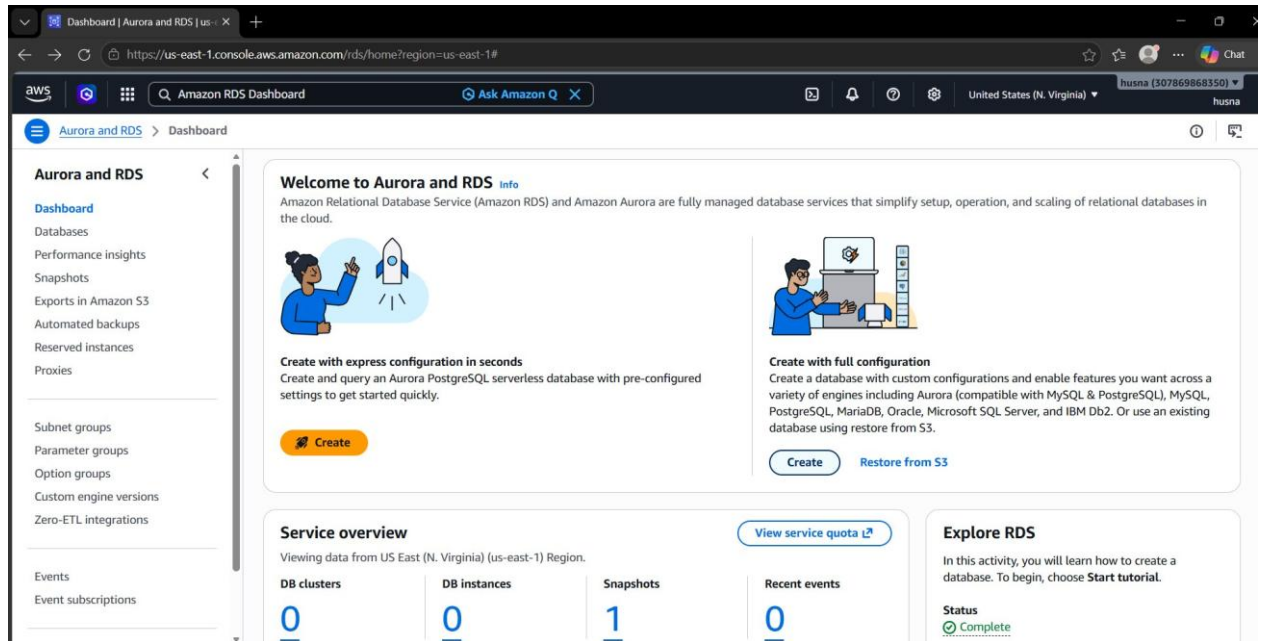


Figure 42: Amazon RDS Dashboard

5.2. Creating a MySQL Database Instance

A new MySQL database instance was created using the Easy Create option. The database was configured with a unique database identifier, administrator username, and password. The Free Tier option was selected to meet the project requirements while minimizing resource usage.

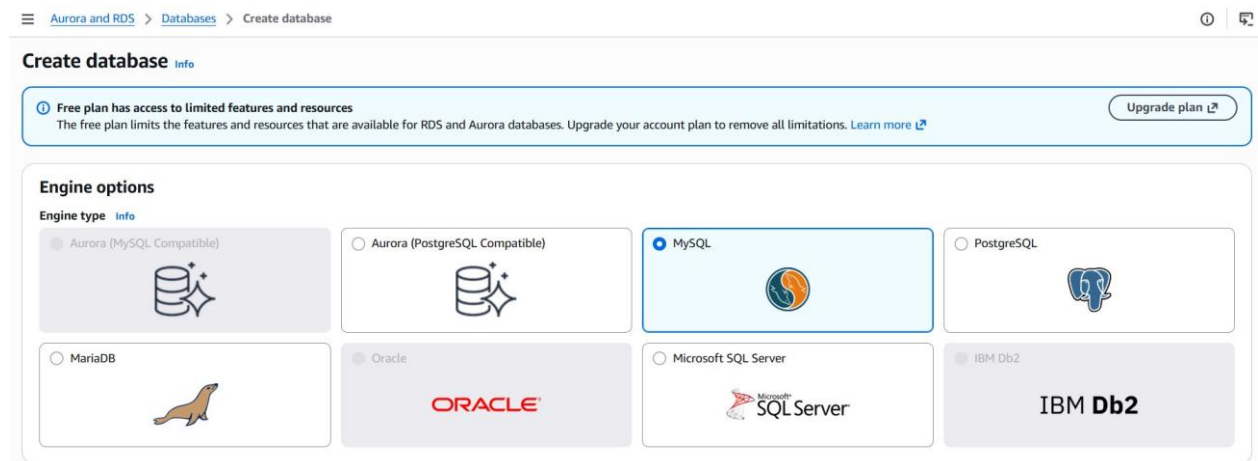
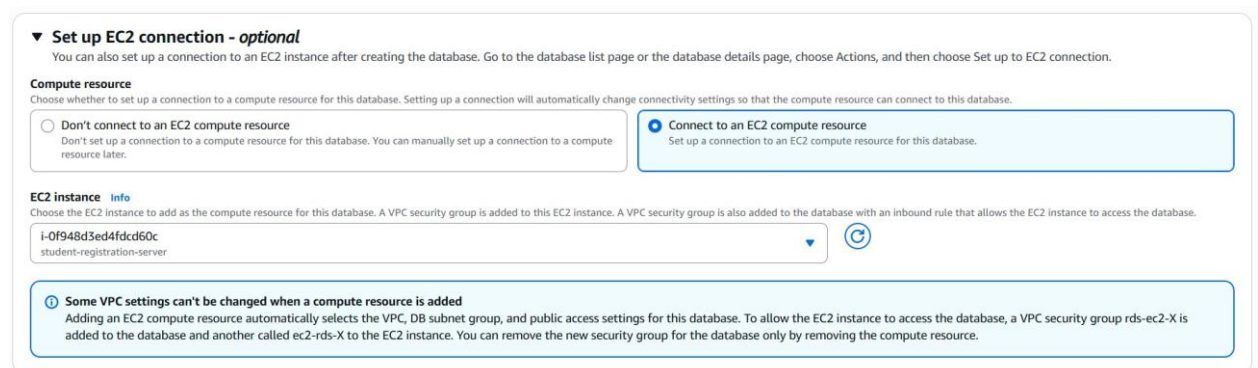


Figure 43: MySQL Database Configuration

5.3. Configuring Database Connectivity

The database instance was connected to the EC2 web server hosting the Student Registration System. This configuration enabled secure communication between the application and the database through the AWS cloud environment.



▼ View default settings for Easy create

Easy create sets the following configurations to their default values, some of which can be changed later. If you want to change any of these settings now, use Standard create.

Configuration ▲	Value	Editable after database is created ▼
Automatic backups	Enabled	Yes
Database port	3306	Yes
DB engine version	8.4.8	Yes
DB instance identifier	student-db	Yes
DB parameter group	default.mysql8.4	Yes
Delete protection	Not enabled	Yes
Encryption	Enabled	No
Maintenance	Auto minor version upgrade enabled	Yes
Monitoring	Not enabled	Yes
Monitoring type	Database Insights - Standard	Yes
Multi-AZ	No	Yes

Option group	default.mysql-8-4	Yes
Performance insights	Not enabled	Yes
Publicly accessible	No	Yes
Subnet group	rds-ec2-db-subnet-group-1	Yes
VPC	student-vpc (vpc-0a95b5cd0775a494f)	No
VPC security group	rds-ec2-1	Yes

ⓘ You are responsible for ensuring that you have all of the necessary rights for any third-party products or services that you use with AWS services.

Cancel
Create database

Figure 44: EC2 and RDS Connectivity Configuration

5.4. Database Deployment

After the configuration was completed, the database instance was deployed and became available for use. Amazon RDS automatically handled the provisioning process and applied the necessary network and security settings.

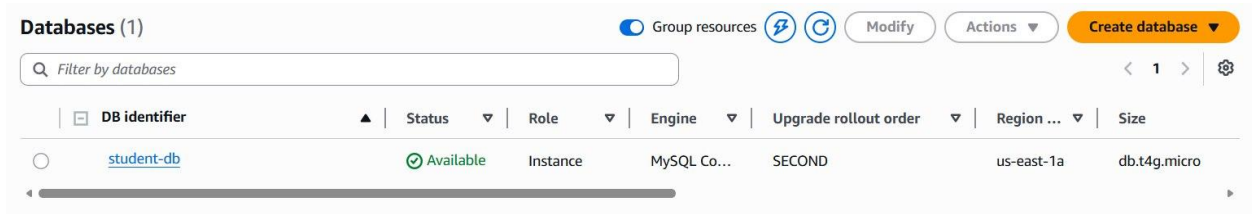


Figure 45: Database Status – Available

5.5. Viewing Database Connectivity Details

The database endpoint, port number, and network configuration were verified to ensure successful connectivity between AWS services.

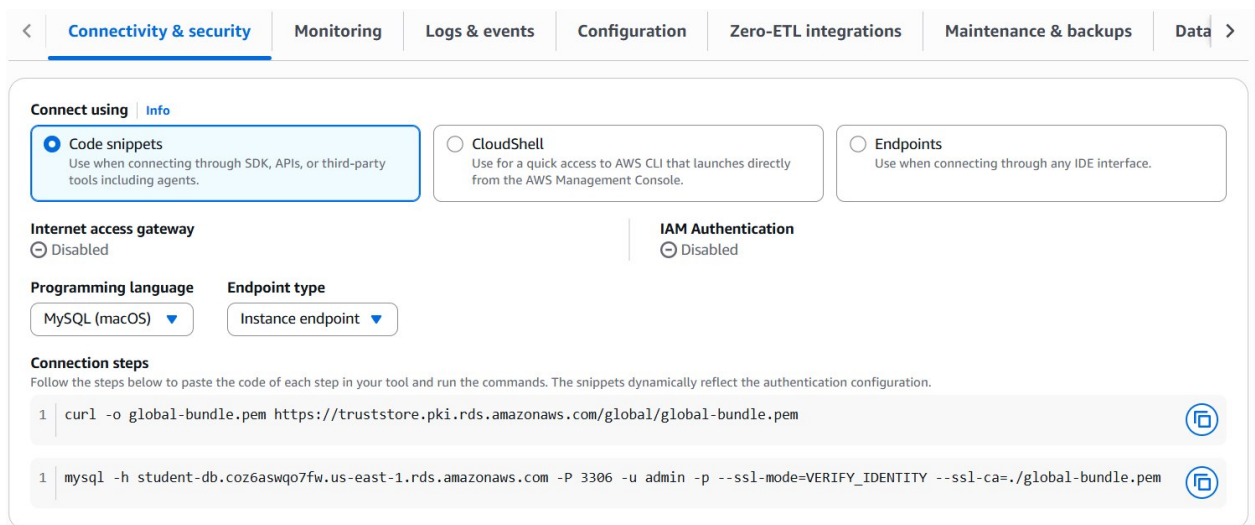
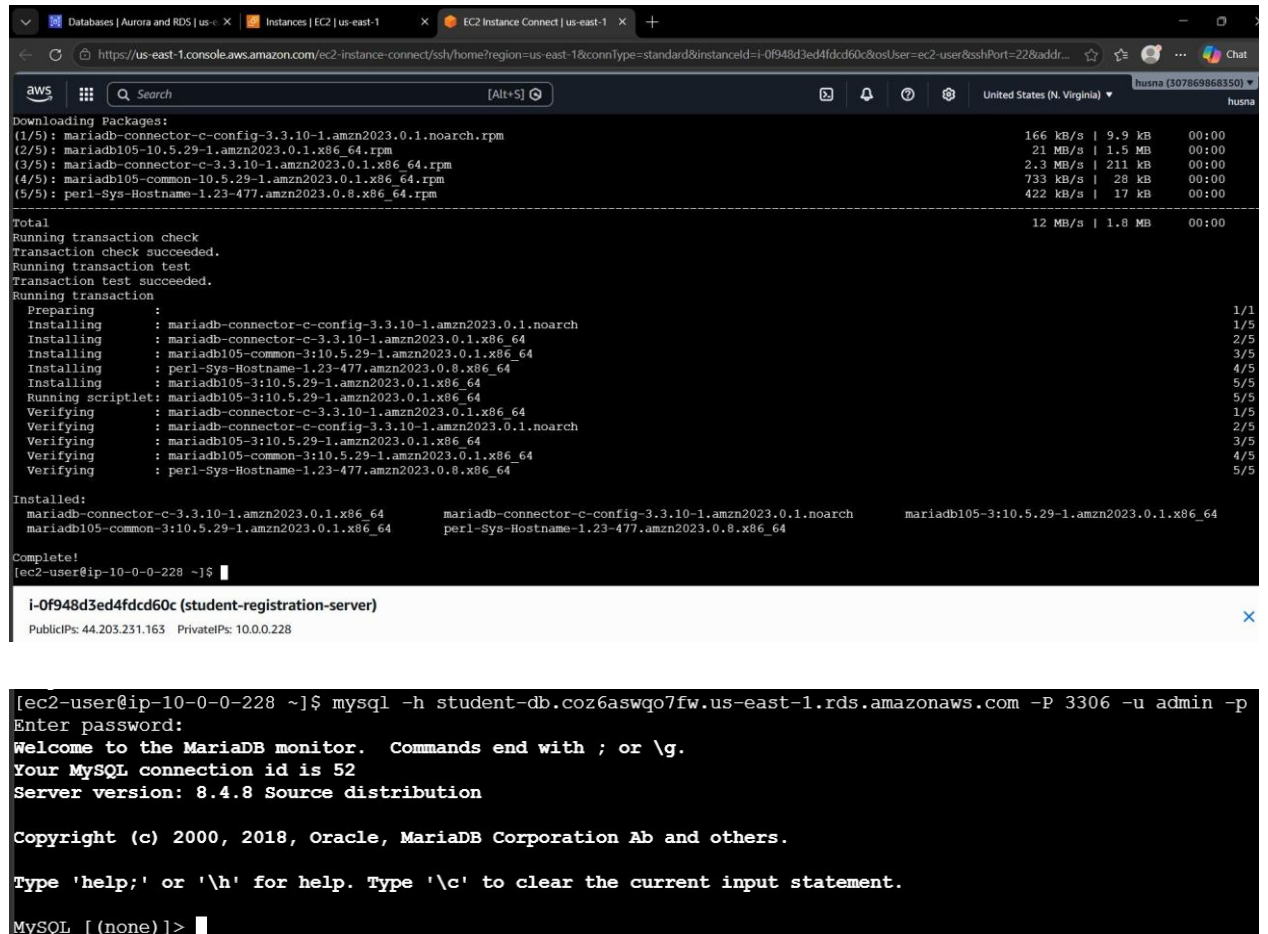


Figure 46: Connectivity and Security Details

5.6. Connecting to the Database

The EC2 instance was used to connect to the MySQL database. The MySQL client was installed on the server, and a connection was established using the database endpoint and administrator credentials.



The screenshot displays the AWS console interface for an EC2 Instance Connect session. The top navigation bar shows the 'Instances | EC2 | us-east-1' tab. The browser address bar indicates the session URL. The main content area shows the progress of downloading and installing packages for the MariaDB connector and common files. A progress table is visible:

Package	Download Speed	Download Size	Progress
(1/5): mariadb-connector-c-config-3.3.10-1.amzn2023.0.1.noarch.rpm	166 kB/s	9.9 kB	00:00
(2/5): mariadb105-10.5.29-1.amzn2023.0.1.x86_64.rpm	21 MB/s	1.5 MB	00:00
(3/5): mariadb-connector-c-3.3.10-1.amzn2023.0.1.x86_64.rpm	2.3 MB/s	211 kB	00:00
(4/5): mariadb105-common-10.5.29-1.amzn2023.0.1.x86_64.rpm	733 kB/s	28 kB	00:00
(5/5): perl-Sys-Hostname-1.23-477.amzn2023.0.8.x86_64.rpm	422 kB/s	17 kB	00:00

Below the progress table, the terminal output shows the installation process:

```
Total 12 MB/s | 1.8 MB 00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing      : 1/1
  Installing     : mariadb-connector-c-config-3.3.10-1.amzn2023.0.1.noarch 1/5
  Installing     : mariadb-connector-c-3.3.10-1.amzn2023.0.1.x86_64 1/5
  Installing     : mariadb105-common-3:10.5.29-1.amzn2023.0.1.x86_64 2/5
  Installing     : perl-Sys-Hostname-1.23-477.amzn2023.0.8.x86_64 3/5
  Installing     : mariadb105-3:10.5.29-1.amzn2023.0.1.x86_64 4/5
  Running scriptlet: mariadb105-3:10.5.29-1.amzn2023.0.1.x86_64 5/5
  Verifying      : mariadb-connector-c-config-3.3.10-1.amzn2023.0.1.noarch 1/5
  Verifying      : mariadb-connector-c-3.3.10-1.amzn2023.0.1.x86_64 2/5
  Verifying      : mariadb105-common-3:10.5.29-1.amzn2023.0.1.x86_64 3/5
  Verifying      : mariadb105-3:10.5.29-1.amzn2023.0.1.x86_64 4/5
  Verifying      : perl-Sys-Hostname-1.23-477.amzn2023.0.8.x86_64 5/5

Installed:
  mariadb-connector-c-3.3.10-1.amzn2023.0.1.x86_64      mariadb-connector-c-config-3.3.10-1.amzn2023.0.1.noarch      mariadb105-3:10.5.29-1.amzn2023.0.1.x86_64
  mariadb105-common-3:10.5.29-1.amzn2023.0.1.x86_64      perl-Sys-Hostname-1.23-477.amzn2023.0.8.x86_64

Complete!
[ec2-user@ip-10-0-0-228 ~]$
```

The bottom part of the screenshot shows the terminal output for the MySQL connection:

```
[ec2-user@ip-10-0-0-228 ~]$ mysql -h student-db.coz6aswqo7fw.us-east-1.rds.amazonaws.com -P 3306 -u admin -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 52
Server version: 8.4.8 Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]>
```

Figure 47: Successful Database Connection

5.7. Creating the Student Database and Table

A database named "studentdb" was created. Subsequently, a table named "students" was developed to store student registration information, including personal details, contact information, course information, and registration records.

```
MySQL [(none)]> CREATE DATABASE studentdb;
Query OK, 1 row affected (0.023 sec)

MySQL [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| studentdb |
| sys |
+-----+
5 rows in set (0.011 sec)

MySQL [(none)]> 
```

```
MySQL [studentdb]> CREATE TABLE students (
    -> student_id INT AUTO_INCREMENT PRIMARY KEY,
    -> student_name VARCHAR(100) NOT NULL,
    -> gender VARCHAR(10),
    -> email VARCHAR(100),
    -> phone_number VARCHAR(20),
    -> address VARCHAR(255),
    -> course VARCHAR(100),
    -> registration_date DATE
    -> );
Query OK, 0 rows affected (0.119 sec)

MySQL [studentdb]> SHOW TABLES;
+-----+
| Tables_in_studentdb |
+-----+
| students |
+-----+
1 row in set (0.006 sec)
```

Figure 48: Student Table Creation

5.8. Inserting and Managing Student Records

Sample student records were inserted into the database to validate its functionality. SQL queries were executed to retrieve, update, and delete records, demonstrating that the database was fully operational and capable of supporting the Student Registration System.

```
MySQL [studentdb]> DESCRIBE students;
```

Field	Type	Null	Key	Default	Extra
student_id	int	NO	PRI	NULL	auto_increment
student_name	varchar(100)	NO		NULL	
gender	varchar(10)	YES		NULL	
email	varchar(100)	YES		NULL	
phone_number	varchar(20)	YES		NULL	
address	varchar(255)	YES		NULL	
course	varchar(100)	YES		NULL	
registration_date	date	YES		NULL	

8 rows in set (0.033 sec)

```
MySQL [studentdb]> INSERT INTO students
```

```
-> (student_name,gender,email,phone_number,address,course,registration_date)
-> VALUES
-> ('Nur Nabilah','Female',
-> 'nabilah@gmail.com','0123456789',
-> 'Kuala Lumpur',
-> 'Software Engineering',
-> '2026-06-01'),
->
-> ('Siti Nurhidayah','Female',
-> 'dayah@yahoo.com','0131234567',
-> 'Selangor',
-> 'Computer Science',
-> '2026-03-15'),
->
-> ('Nur Husna','Female',
-> 'husna@gmail.com','0141234567',
-> 'Penang',
-> 'Information Technology',
-> '2026-08-06'),
->
-> ('Farrah Syahirah','Female',
-> 'farrah@yahoo.com','0193567341',
-> 'Pahang',
-> 'Internet Of Things',
-> '2026-12-26');
```

```
Query OK, 4 rows affected (0.029 sec)
```

```
Records: 4 Duplicates: 0 Warnings: 0
```

```
MySQL [studentdb]> SELECT * FROM students;
```

student_id	student_name	gender	email	phone_number	address	course	registration_date
1	Nur Nabilah	Female	nabilah@gmail.com	0123456789	Kuala Lumpur	Software Engineering	2026-06-01
2	Siti Nurhidayah	Female	dayah@yahoo.com	0131234567	Selangor	Computer Science	2026-03-15
3	Nur Husna	Female	husna@gmail.com	0141234567	Penang	Information Technology	2026-08-06
4	Farrah Syahirah	Female	farrah@yahoo.com	0193567341	Pahang	Internet Of Things	2026-12-26

4 rows in set (0.002 sec)

```

MySQL [studentdb]> UPDATE students
  -> SET course='Data Science'
  -> WHERE student_id=1;
Query OK, 1 row affected (0.023 sec)
Rows matched: 1  Changed: 1  Warnings: 0

MySQL [studentdb]> SELECT * FROM students;
+-----+-----+-----+-----+-----+-----+-----+-----+
| student_id | student_name | gender | email | phone_number | address | course | registration_date |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | Nur Nabilah | Female | nabilah@gmail.com | 0123456789 | Kuala Lumpur | Data Science | 2026-06-01 |
| 2 | Siti Nurhidayah | Female | dayah@yahoo.com | 0131234567 | Selangor | Computer Science | 2026-03-15 |
| 3 | Nur Husna | Female | husna@gmail.com | 0141234567 | Penang | Information Technology | 2026-08-06 |
| 4 | Farrah Syahirah | Female | farrah@yahoo.com | 0193567341 | Pahang | Internet Of Things | 2026-12-26 |
+-----+-----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.003 sec)

MySQL [studentdb]> DELETE FROM students
  -> WHERE student_id=1;
Query OK, 1 row affected (0.041 sec)

MySQL [studentdb]> SELECT * FROM students;
+-----+-----+-----+-----+-----+-----+-----+-----+
| student_id | student_name | gender | email | phone_number | address | course | registration_date |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 2 | Siti Nurhidayah | Female | dayah@yahoo.com | 0131234567 | Selangor | Computer Science | 2026-03-15 |
| 3 | Nur Husna | Female | husna@gmail.com | 0141234567 | Penang | Information Technology | 2026-08-06 |
| 4 | Farrah Syahirah | Female | farrah@yahoo.com | 0193567341 | Pahang | Cyber Security | 2026-12-26 |
+-----+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.001 sec)

MySQL [studentdb]>

```

Figure 49: Student Records Stored in Database

The successful implementation of Amazon RDS provided a secure, scalable, and reliable database platform for the Student Registration System. The database supports efficient storage and management of student information while integrating seamlessly with other AWS services used throughout the project.

6. Testing and Validation

To verify the functionality of the Student Registration System, several tests were conducted after deployment

Test Case	Expected Result	Actual Result	Status
Access website through EC2 Public IP	Website loads successfully	Website displayed correctly	Pass
Submit student registration data	Data stored successfully	Record inserted into database	Pass
Retrieve student information	Data displayed correctly	Information retrieved successfully	Pass
Upload project files to S3	Files stored in bucket	Files uploaded successfully	Pass
Connect EC2 with RDS	Successful database communication	Connection established	Pass
Access website through Internet Gateway	Website reachable from internet	Website accessible	Pass

Based on the testing results, all major system functions operated successfully and met the project requirements.

7. Project Outcome

The AWS Cloud-Based Student Registration System was successfully implemented and deployed using Amazon Web Services.

The project successfully demonstrated the integration of multiple AWS services, including Amazon EC2 for web hosting, Amazon S3 for storage, Amazon VPC for networking and security, and Amazon RDS for database management.

Users are able to access the web application, submit student information, and retrieve records through a cloud-based environment. The networking infrastructure provides secure communication between services while ensuring internet accessibility for users.

The project achieved all objectives and demonstrated practical implementation of cloud computing concepts learned throughout the course.

8. References

Amazon Web Services. (2025). Amazon EC2 Documentation. Available at:

<https://docs.aws.amazon.com/ec2/>

Amazon Web Services. (2025). Amazon S3 Documentation. Available at:

<https://docs.aws.amazon.com/s3/>

Amazon Web Services. (2025). Amazon VPC Documentation. Available at:

<https://docs.aws.amazon.com/vpc/>

Amazon Web Services. (2025). Amazon RDS Documentation. Available at:

<https://docs.aws.amazon.com/rds/>

Amazon Web Services. (2025). AWS Getting Started Guide. Available at:

<https://docs.aws.amazon.com/>

9. Appendix

- GitHub Repository Link:
- Youtube Presentation Link:
https://youtu.be/4bmhvCM_gAw?si=n9y8TJWRISH8ARQB